

Notebook Link for examination:

https://colab.research.google.com/drive/1aMWRczZnLH_QCgay2eQObcaks-ADkPqm#scrollTo=3eznFBruhGD4

⚠ Note on Interactive Plots in Colab

Due to the heavy resource requirements of WebGL-based 3D/interactive visualizations, Plotly figures may occasionally fail to render during a 'Run All' execution.

If a graph is not visible:

1. Please **manually rerun** the specific code cell after running all cells first.
2. Interact with the graphs wisely, as multiple heavy plots can strain the browser's UI context.
3. If 'Sad Face' icons appear (means disconnection), it is recommended to use manual **rerun-cell method** to get the interactive plot again.

Mistakes to avoid:

1. Open/close of gemini panel.
2. Resizing of the colab notebook by checking the 'Files', 'Data explorer/inspector', 'Secrets', etc.
3. Scrolling away from the interactive plotly plot's cell.
4. Any other means that affect the width or your interaction time with this notebook
 - Leads to this **Colab UI vs Plotly disconnection** while showing the interactive plots (Metric Scale will still be shown in some cases proving the successful execution of the code)

Best Solution:

Steps to run the research notebook:

1. Press Run All
2. Re-run the cell where the plot is not visible (**often partially: main content**)

Sorry For the inconvenience that will cause due to heavy load graphs but it's worth spending time with graphs for research & examination purposes.

```
!pip install scikit-learn plotly numpy pandas --quiet
!pip install --upgrade plotly --quiet
```

#Forecasting Methodology (Step-By-Step)

1. Data Ingestion & Structuring

```
import pandas as pd
```

```
url="https://drive.google.com/uc?
export=download&id=1xZo782T4EfnkC0BmCwJTb0DZYYHo0XKm"
```

```

# Load daily time-series data
df = pd.read_csv(url)

# Convert 'Date' to datetime format, coercing errors to NaT
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')

# Drop rows where 'Date' is NaT (invalid dates) which can cause issues
with indexing
df.dropna(subset=['Date'], inplace=True)

# Identify numerical columns that might be read as strings due to
commas or other non-numeric chars
numeric_cols = [
    'Children apprehended and placed in CBP custody*',
    'Children in CBP custody',
    'Children transferred out of CBP custody',
    'Children in HHS Care',
    'Children discharged from HHS Care'
]

# Convert identified columns to numeric, handling commas and coercing
errors to NaN
for col in numeric_cols:
    # Convert to string first to handle mixed types gracefully, then
    replace commas, then convert to numeric
    df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', ''),
errors='coerce')

# Ensure chronological ordering
df = df.sort_values(by='Date')

# Handle duplicate dates by grouping by 'Date' and summing numerical
columns.
# This ensures each date has a single entry before setting it as an
index.
if not df['Date'].is_unique:
    print("Duplicate dates found after initial cleaning. Aggregating
data by summing numeric columns.")
    # Group by 'Date' and sum only numeric columns, then reset index
to make 'Date' a column again
    df = df.groupby('Date').sum(numeric_only=True).reset_index()

# Set Date as index for easier time-series operations
df = df.set_index('Date')

# Create a complete daily index for the entire range and reindex the
DataFrame.
# This fills in any missing dates with NaNs (which will be handled by
fillna(0) later).
full_date_range = pd.date_range(start=df.index.min(),

```

```
end=df.index.max(), freq='D')
df = df.reindex(full_date_range)
```

```
display(df.head())
display(df.info())
display(df.describe())
```

```
-----
-----
SSLError                                Traceback (most recent call
last)
```

```
Cell In[2], line 5
```

```
3 url="https://drive.google.com/uc?
export=download&id=1xZo782T4EfnkC0BmCwJTb0DZYYHo0XKm"
4 # Load daily time-series data
----> 5 df = pd.read_csv(url)
7 # Convert 'Date' to datetime format, coercing errors to NaT
8 df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

```
File c:\AnacondaEnv\ukvenv\lib\site-packages\pandas\io\parsers\
readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter,
header, names, index_col, usecols, dtype, engine, converters,
true_values, false_values, skipinitialspace, skiprows, skipfooter,
nrows, na_values, keep_default_na, na_filter, verbose,
skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col,
date_parser, date_format, dayfirst, cache_dates, iterator, chunksize,
compression, thousands, decimal, lineterminator, quotechar, quoting,
doublequote, escapechar, comment, encoding, encoding_errors, dialect,
on_bad_lines, delim_whitespace, low_memory, memory_map,
float_precision, storage_options, dtype_backend)
```

```
1013 kwds_defaults = _refine_defaults_read(
1014     dialect,
1015     delimiter,
1016     (...)
1022     dtype_backend=dtype_backend,
1023 )
1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)
```

```
File c:\AnacondaEnv\ukvenv\lib\site-packages\pandas\io\parsers\
readers.py:620, in _read(filepath_or_buffer, kwds)
```

```
617 _validate_names(kwds.get("names", None))
619 # Create the parser.
--> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
622 if chunksize or iterator:
623     return parser
```

```
File c:\AnacondaEnv\ukvenv\lib\site-packages\pandas\io\parsers\
readers.py:1620, in TextFileReader.__init__(self, f, engine, **kwds)
1617     self.options["has_index_names"] = kwds["has_index_names"]
```

```
1619 self.handles: IOHandles | None = None
-> 1620 self._engine = self._make_engine(f, self.engine)
```

File c:\AnacondaEnv\ukvenv\lib\site-packages\pandas\io\parsers\readers.py:1880, in TextFileReader._make_engine(self, f, engine)

```
1878     if "b" not in mode:
1879         mode += "b"
-> 1880 self.handles = get_handle(
1881     f,
1882     mode,
1883     encoding=self.options.get("encoding", None),
1884     compression=self.options.get("compression", None),
1885     memory_map=self.options.get("memory_map", False),
1886     is_text=is_text,
1887     errors=self.options.get("encoding_errors", "strict"),
1888     storage_options=self.options.get("storage_options", None),
1889 )
1890 assert self.handles is not None
1891 f = self.handles.handle
```

File c:\AnacondaEnv\ukvenv\lib\site-packages\pandas\io\common.py:728, in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_options)

```
725     codecs.lookup_error(errors)
727 # open URLs
--> 728 ioargs = _get_filepath_or_buffer(
729     path_or_buf,
730     encoding=encoding,
731     compression=compression,
732     mode=mode,
733     storage_options=storage_options,
734 )
736 handle = ioargs.filepath_or_buffer
737 handles: list[BaseBuffer]
```

File c:\AnacondaEnv\ukvenv\lib\site-packages\pandas\io\common.py:384, in _get_filepath_or_buffer(filepath_or_buffer, encoding, compression, mode, storage_options)

```
382 # assuming storage_options is to be interpreted as headers
383 req_info = urllib.request.Request(filepath_or_buffer,
headers=storage_options)
-> 384 with urlopen(req_info) as req:
385     content_encoding = req.headers.get("Content-Encoding",
None)
386     if content_encoding == "gzip":
387         # Override compression based on Content-Encoding
header
```

File c:\AnacondaEnv\ukvenv\lib\site-packages\pandas\io\common.py:289, in urlopen(*args, **kwargs)

```

283 """
284 Lazy-import wrapper for stdlib urlopen, as that imports a big
chunk of
285 the stdlib.
286 """
287 import urllib.request
--> 289 return urllib.request.urlopen(*args, **kwargs)

```

File c:\AnacondaEnv\ukvenv\lib\urllib\request.py:216, in urlopen(url, data, timeout, cafile, capath, cadefault, context)

```

214 else:
215     opener = _opener
--> 216 return opener.open(url, data, timeout)

```

File c:\AnacondaEnv\ukvenv\lib\urllib\request.py:519, in OpenerDirector.open(self, fullurl, data, timeout)

```

516     req = meth(req)
518 sys.audit('urllib.Request', req.full_url, req.data,
req.headers, req.get_method())
--> 519 response = self._open(req, data)
521 # post-process response
522 meth_name = protocol+"_response"

```

File c:\AnacondaEnv\ukvenv\lib\urllib\request.py:536, in OpenerDirector._open(self, req, data)

```

533     return result
535 protocol = req.type
--> 536 result = self._call_chain(self.handle_open, protocol, protocol
+
537                               '_open', req)
538 if result:
539     return result

```

File c:\AnacondaEnv\ukvenv\lib\urllib\request.py:496, in OpenerDirector._call_chain(self, chain, kind, meth_name, *args)

```

494 for handler in handlers:
495     func = getattr(handler, meth_name)
--> 496     result = func(*args)
497     if result is not None:
498         return result

```

File c:\AnacondaEnv\ukvenv\lib\urllib\request.py:1391, in HTTPSHandler.https_open(self, req)

```

1390 def https_open(self, req):
-> 1391     return self.do_open(http.client.HTTPSConnection, req,
1392                           context=self._context,
check_hostname=self._check_hostname)

```

File c:\AnacondaEnv\ukvenv\lib\urllib\request.py:1317, in AbstractHTTPHandler.do_open(self, http_class, req, **http_conn_args)

```
1314     raise URLError('no host given')
1316 # will parse host:port
-> 1317 h = http_class(host, timeout=req.timeout, **http_conn_args)
1318 h.set_debuglevel(self._debuglevel)
1320 headers = dict(req.unredirected_hdrs)
```

File c:\AnacondaEnv\ukvenv\lib\http\client.py:1442, in
HTTPSConnection.__init__(self, host, port, key_file, cert_file,
timeout, source_address, context, check_hostname, blocksize)

```
1440 self.cert_file = cert_file
1441 if context is None:
-> 1442     context = ssl._create_default_https_context()
1443     # send ALPN extension to indicate HTTP/1.1 protocol
1444     if self._http_vsn == 11:
```

File c:\AnacondaEnv\ukvenv\lib\ssl.py:771, in
create_default_context(purpose, cafile, capath, cadata)
766 context.load_verify_locations(cafile, capath, cadata)
767 elif context.verify_mode != CERT_NONE:
768 # no explicit cafile, capath or cadata but the verify mode
is

```
769     # CERT_OPTIONAL or CERT_REQUIRED. Let's try to load
default system
770     # root CA certificates for the given purpose. This may
fail silently.
--> 771     context.load_default_certs(purpose)
772 # OpenSSL 1.1.1 keylog file
773 if hasattr(context, 'keylog_filename'):
```

File c:\AnacondaEnv\ukvenv\lib\ssl.py:592, in
SSLContext.load_default_certs(self, purpose)
590 if sys.platform == "win32":
591 for storename in self._windows_cert_stores:
--> 592 self._load_windows_store_certs(storename, purpose)
593 self.set_default_verify_paths()

File c:\AnacondaEnv\ukvenv\lib\ssl.py:584, in
SSLContext._load_windows_store_certs(self, storename, purpose)
582 warnings.warn("unable to enumerate Windows certificate
store")
583 if certs:
--> 584 self.load_verify_locations(cadata=certs)
585 return certs

SSLError: [ASN1: NOT_ENOUGH_DATA] not enough data (_ssl.c:4040)

2. Data Quality & Validation

```
# Identify missing or duplicated dates
missing_dates = df.index[df.isnull().all(axis=1)].tolist()
duplicated_dates = df.index[df.index.duplicated()].tolist()

print(f"Missing dates: {missing_dates}")
print(f"Duplicated dates: {duplicated_dates}")

# Fill missing values for numerical columns (assuming forward fill or
# 0 based on context)
# For this dataset, it might be better to fill with 0 or the previous
# day's value if appropriate for 'load' metrics.
# Let's use forward fill for now for 'load' type columns, and 0 for
# 'flow' type columns if a day is entirely missing.
# However, if an entire date is missing, all values will be NaN as per
# reindex.
# For simplicity, let's fill NaNs in numerical columns with 0 after
# reindexing, as it implies no activity/load on that day.
df = df.fillna(0)

# Validate logical constraints
# Transfers <= CBP custody
constraint_1_violations = df[df['Children transferred out of CBP
custody'] > df['Children in CBP custody']]
print(f"\nViolations: Transfers > CBP custody:\n{constraint_1_violations}")

# Discharges <= HHS care
constraint_2_violations = df[df['Children discharged from HHS Care'] >
df['Children in HHS Care']]
print(f"\nViolations: Discharges > HHS care:\n{constraint_2_violations}")

# Flag reporting anomalies for transparency (these are the violations
# identified above)
if not constraint_1_violations.empty:
    print("\nAnomaly detected: 'Children transferred out of CBP
custody' is greater than 'Children in CBP custody' on some dates.")
if not constraint_2_violations.empty:
    print("\nAnomaly detected: 'Children discharged from HHS Care' is
greater than 'Children in HHS Care' on some dates.")

display(df.head())

Missing dates: [Timestamp('2023-01-13 00:00:00'), Timestamp('2023-01-
14 00:00:00'), Timestamp('2023-01-15 00:00:00'), Timestamp('2023-01-16
00:00:00'), Timestamp('2023-01-17 00:00:00'), Timestamp('2023-01-18
00:00:00'), Timestamp('2023-01-19 00:00:00'), Timestamp('2023-01-20
00:00:00'), Timestamp('2023-01-21 00:00:00'), Timestamp('2023-01-26
```

[illegible]

[illegible]

[illegible]

```

00:00:00'), Timestamp('2025-07-18 00:00:00'), Timestamp('2025-07-19
00:00:00'), Timestamp('2025-07-20 00:00:00'), Timestamp('2025-07-25
00:00:00'), Timestamp('2025-07-26 00:00:00'), Timestamp('2025-08-01
00:00:00'), Timestamp('2025-08-02 00:00:00'), Timestamp('2025-08-08
00:00:00'), Timestamp('2025-08-09 00:00:00'), Timestamp('2025-08-15
00:00:00'), Timestamp('2025-08-16 00:00:00'), Timestamp('2025-08-22
00:00:00'), Timestamp('2025-08-23 00:00:00'), Timestamp('2025-08-29
00:00:00'), Timestamp('2025-08-30 00:00:00'), Timestamp('2025-08-31
00:00:00'), Timestamp('2025-09-05 00:00:00'), Timestamp('2025-09-06
00:00:00'), Timestamp('2025-09-12 00:00:00'), Timestamp('2025-09-13
00:00:00'), Timestamp('2025-09-19 00:00:00'), Timestamp('2025-09-20
00:00:00'), Timestamp('2025-09-26 00:00:00'), Timestamp('2025-09-27
00:00:00'), Timestamp('2025-10-03 00:00:00'), Timestamp('2025-10-04
00:00:00'), Timestamp('2025-10-10 00:00:00'), Timestamp('2025-10-11
00:00:00'), Timestamp('2025-10-12 00:00:00'), Timestamp('2025-10-17
00:00:00'), Timestamp('2025-10-18 00:00:00'), Timestamp('2025-10-24
00:00:00'), Timestamp('2025-10-25 00:00:00'), Timestamp('2025-10-31
00:00:00'), Timestamp('2025-11-01 00:00:00'), Timestamp('2025-11-07
00:00:00'), Timestamp('2025-11-08 00:00:00'), Timestamp('2025-11-10
00:00:00'), Timestamp('2025-11-14 00:00:00'), Timestamp('2025-11-15
00:00:00'), Timestamp('2025-11-21 00:00:00'), Timestamp('2025-11-22
00:00:00'), Timestamp('2025-11-26 00:00:00'), Timestamp('2025-11-28
00:00:00'), Timestamp('2025-11-29 00:00:00'), Timestamp('2025-12-05
00:00:00'), Timestamp('2025-12-06 00:00:00'), Timestamp('2025-12-12
00:00:00'), Timestamp('2025-12-13 00:00:00'), Timestamp('2025-12-19
00:00:00'), Timestamp('2025-12-20 00:00:00')]

```

Duplicated dates: []

Violations: Transfers > CBP custody:

Children apprehended and placed in CBP custody*	\
2023-01-24	47.0
2023-01-25	20.0
2023-02-02	15.0
2023-02-22	107.0
2023-02-23	101.0
...	...
2025-01-30	47.0
2025-02-02	20.0
2025-02-09	15.0
2025-02-13	15.0
2025-02-27	10.0

Children in CBP custody	Children transferred out of CBP custody \
2023-01-24	42.0
47.0	
2023-01-25	22.0
41.0	
2023-02-02	13.0

23.0	
2023-02-22	215.0
230.0	
2023-02-23	162.0
178.0	
...	...
...	
2025-01-30	42.0
47.0	
2025-02-02	22.0
41.0	
2025-02-09	13.0
23.0	
2025-02-13	10.0
23.0	
2025-02-27	15.0
17.0	

	Children in HHS Care	Children discharged from HHS Care
2023-01-24	7433.0	175.0
2023-01-25	7538.0	180.0
2023-02-02	7879.0	298.0
2023-02-22	7978.0	232.0
2023-02-23	7914.0	386.0
...	...	...
2025-01-30	3923.0	159.0
2025-02-02	3483.0	168.0
2025-02-09	2878.0	99.0
2025-02-13	2703.0	72.0
2025-02-27	2391.0	37.0

[86 rows x 5 columns]

Violations: Discharges > HHS care:

Empty DataFrame

Columns: [Children apprehended and placed in CBP custody*, Children in CBP custody, Children transferred out of CBP custody, Children in HHS Care, Children discharged from HHS Care]

Index: []

Anomaly detected: 'Children transferred out of CBP custody' is greater than 'Children in CBP custody' on some dates.

```
{
  "summary": {
    "name": "display(df",
    "rows": 5,
    "fields": [
      {
        "column": "Children apprehended and placed in CBP custody*",
        "properties": {
          "dtype": "number",
          "std": 14.75804865149861,
          "min": 0.0,
          "max": 33.0,
          "num_unique_values": 2,
          "samples": [0.0, 33.0]
        },
        "semantic_type": "\"",
        "description": "\"\"\""
      }
    ]
  }
}
```

```

n    },\n    {\n        \"column\": \"Children in CBP custody\",\n        \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 23.70232056149777,\n            \"min\": 0.0,\n            \"max\": 53.0,\n            \"num_unique_values\": 2,\n            \"samples\": [\n                0.0,\n                53.0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        },\n        {\n            \"column\": \"Children transferred out of CBP custody\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 15.20526224699857,\n                \"min\": 0.0,\n                \"max\": 34.0,\n                \"num_unique_values\": 2,\n                \"samples\": [\n                    0.0,\n                    34.0\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n            },\n            {\n                \"column\": \"Children in HHS Care\",\n                \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 2936.4044680527245,\n                    \"min\": 0.0,\n                    \"max\": 6566.0,\n                    \"num_unique_values\": 2,\n                    \"samples\": [\n                        0.0,\n                        6566.0\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\"\n                },\n                {\n                    \"column\": \"Children discharged from HHS Care\",\n                    \"properties\": {\n                        \"dtype\": \"number\",\n                        \"std\": 194.98512763798166,\n                        \"min\": 0.0,\n                        \"max\": 436.0,\n                        \"num_unique_values\": 2,\n                        \"samples\": [\n                            0.0,\n                            436.0\n                        ],\n                        \"semantic_type\": \"\",\n                        \"description\": \"\"\n                    },\n                    }\n                }\n            ],\n            \"type\": \"dataframe\"

```

2.1 Individual Trends: Children Metrics

This section visualizes the daily activity for each key metric regarding children in custody and care.

```

import plotly.express as px
from IPython.display import HTML, display

children_metrics = [
    'Children apprehended and placed in CBP custody*',
    'Children in CBP custody',
    'Children transferred out of CBP custody',
    'Children in HHS Care',
    'Children discharged from HHS Care'
]

# Loop through each metric and create interactive line charts
for metric in children_metrics:
    fig = px.line(
        df,
        x=df.index,
        y=metric,
        title=f'Daily Trend: {metric}',
        labels={'x': 'Date', 'y': 'Count'},
        line_shape='linear'
    )

```

```

)
fig.update_traces(line=dict(color='teal'))
fig.update_layout(
    template='plotly_white',
    xaxis_title='Date',
    yaxis_title='Count',
    hovermode='x unified',
    autosize=True,
    title=dict(x=0.5, xanchor='center') # Center the title
)
#fig.show()
#display(HTML(fig.to_html()))
display(fig)

```

2.2 Comparative Analysis: Children Metrics

The following chart overlays all children-related metrics to identify correlations and lead/lag relationships between CBP custody and HHS care.

3D Visualization: Children Metrics Evolution

This interactive 3D plot separates each metric along the Z-axis, allowing you to see temporal correlations and the magnitude of different metrics without the overlap seen in 2D line charts.

```

import plotly.graph_objects as go
import pandas as pd
import numpy as np
import plotly.express as px
from IPython.display import display

# Force 'colab' renderer and include the JavaScript library explicitly
import plotly.io as pio
pio.renderers.default = "colab"

# Prepare data for plotting
children_metrics = [
    'Children apprehended and placed in CBP custody*',
    'Children in CBP custody',
    'Children transferred out of CBP custody',
    'Children in HHS Care',
    'Children discharged from HHS Care'
]

# Descriptive labels for the Y-axis segments
metric_labels = [
    'Apprehended (CBP)',
    'In CBP Custody',
    'Transferred out (CBP)',
    'In HHS Care',

```

```

        'Discharged (HHS)'
    ]

    # Use a vibrant discrete color palette
    vibrant_colors = px.colors.qualitative.Bold

    fig_3d_flow = go.Figure()

    # Map metrics to Z-axis indices
    for i, metric in enumerate(children_metrics):
        # Filter out NaNs if any
        plot_df = df[[metric]].dropna()

        fig_3d_flow.add_trace(go.Scatter3d(
            x=plot_df.index,
            y=[i] * len(plot_df),
            z=plot_df[metric],
            mode='lines+markers',
            name=metric_labels[i],
            line=dict(width=6, color=vibrant_colors[i %
len(vibrant_colors)]),
            marker=dict(size=2, opacity=0.5)
        ))

    fig_3d_flow.update_layout(
        title=dict(
            text='<b>3D Consolidated Flow: Children Metrics Over
Time</b>',
            x=0.5,
            xanchor='center', yanchor='top',
            font=dict(size=17)
        ),
        scene=dict(
            xaxis_title='<b>Date</b>',
            yaxis_title='<b>Metric Segment</b>',
            zaxis_title='<b>Children</b>',
            xaxis=dict(gridcolor='rgb(200, 200, 200)',
backgroundcolor='rgb(230, 230,230)',
                tickfont=dict(size=10), title_font=dict(size=12)),
            yaxis=dict(
                tickmode='array',
                tickvals=list(range(len(children_metrics))),
                ticktext=metric_labels,
                backgroundcolor='rgb(220, 220, 220)',
                tickfont=dict(size=10), title_font=dict(size=12)
            ),
            zaxis=dict(backgroundcolor='rgb(240, 240, 240)',
                tickfont=dict(size=10), title_font=dict(size=12))
        ),
        margin=dict(l=0, r=0, b=0, t=50),

```

```

    paper_bgcolor='white',
    plot_bgcolor='white',
    height=700
)

# Explicitly call the colab renderer to fix the 'localhost'/'sad face'
icon issue
fig_3d_flow.show(renderer="colab")

print("3D Visualization re-rendered with explicit Colab renderer.")
3D Visualization re-rendered with explicit Colab renderer.

```

3. Derived Healthcare Capacity Metrics

```

import numpy as np

# Total System Load: CBP Custody + HHS Care
df['Total System Load'] = df['Children in CBP custody'] + df['Children
in HHS Care']

# Net Daily Intake: Transfers into HHS - Discharges from HHS
df['Net Daily Intake'] = df['Children transferred out of CBP custody']
- df['Children discharged from HHS Care']

# Care Load Growth Rate: Day-over-day percentage change in Total
System Load
df['Care Load Growth Rate'] = df['Total System Load'].pct_change() *
100 # as a percentage
df['Care Load Growth Rate'] = df['Care Load Growth
Rate'].replace([np.inf, -np.inf], np.nan) # Handle inf values without
inplace

# Backlog Indicator: Sustained positive net intake over time (This is
conceptual and would require further analysis or a specific threshold)
# For now, we can create a simple indicator if Net Daily Intake is
positive.
df['Positive Net Intake'] = (df['Net Daily Intake'] > 0).astype(int)

display(df.head())

{"summary": "{\n  \"name\": \"display(df\", \n  \"rows\": 5, \n
\"fields\": [\n    {\n      \"column\": \"Children apprehended and
placed in CBP custody*\", \n      \"properties\": {\n        \"dtype\":
\"number\", \n        \"std\": 14.75804865149861, \n        \"min\":
0.0, \n        \"max\": 33.0, \n        \"num_unique_values\": 2, \n
\"samples\": [\n          0.0, \n          33.0 \n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n
    }, \n    {\n      \"column\": \"Children in CBP custody\", \n
\"properties\": {\n        \"dtype\": \"number\", \n        \"std\":

```



```

23.70232056149777,\n          \"min\": 0.0,\n          \"max\": 53.0,\n          \"num_unique_values\": 2,\n          \"samples\": [\n              0.0,\n              53.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\":\n          \"Children transferred out of CBP custody\",\n          \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 15.20526224699857,\n              \"min\": 0.0,\n              \"max\": 34.0,\n              \"num_unique_values\":\n              2,\n              \"samples\": [\n                  0.0,\n                  34.0\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\",\n              \"column\":\n              \"Children in HHS Care\",\n              \"properties\": {\n                  \"dtype\":\n                  \"number\",\n                  \"std\": 2936.4044680527245,\n                  \"min\":\n                  0.0,\n                  \"max\": 6566.0,\n                  \"num_unique_values\": 2,\n                  \"samples\": [\n                      0.0,\n                      6566.0\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\",\n                  \"column\":\n                  \"Children discharged from HHS\n                  Care\",\n                  \"properties\": {\n                      \"dtype\": \"number\",\n                      \"std\": 194.98512763798166,\n                      \"min\": 0.0,\n                      \"max\":\n                      436.0,\n                      \"num_unique_values\": 2,\n                      \"samples\": [\n                          0.0,\n                          436.0\n                      ],\n                      \"semantic_type\": \"\",\n                      \"description\": \"\",\n                      \"column\":\n                      \"Total System Load\",\n                      \"properties\": {\n                          \"dtype\":\n                          \"number\",\n                          \"std\": 2960.106788614222,\n                          \"min\":\n                          0.0,\n                          \"max\": 6619.0,\n                          \"num_unique_values\": 2,\n                          \"samples\": [\n                              0.0,\n                              6619.0\n                          ],\n                          \"semantic_type\": \"\",\n                          \"description\": \"\",\n                          \"column\":\n                          \"Net Daily Intake\",\n                          \"properties\": {\n                              \"dtype\": \"number\",\n                              \"std\":\n                              179.7798653909831,\n                              \"min\": -402.0,\n                              \"max\": 0.0,\n                              \"num_unique_values\": 2,\n                              \"samples\": [\n                                  0.0,\n                                  -402.0\n                              ],\n                              \"semantic_type\": \"\",\n                              \"description\": \"\",\n                              \"column\":\n                              \"Care Load Growth Rate\",\n                              \"properties\": {\n                                  \"dtype\": \"number\",\n                                  \"std\": null,\n                                  \"min\": -\n                                  100.0,\n                                  \"max\": -100.0,\n                                  \"num_unique_values\": 1,\n                                  \"samples\": [\n                                      -100.0\n                                  ],\n                                  \"semantic_type\": \"\",\n                                  \"description\": \"\",\n                                  \"column\":\n                                  \"Positive Net Intake\",\n                                  \"properties\": {\n                                      \"dtype\": \"number\",\n                                      \"std\":\n                                      0,\n                                      \"min\": 0,\n                                      \"max\": 0,\n                                      \"num_unique_values\": 1,\n                                      \"samples\": [\n                                          0\n                                      ],\n                                      \"semantic_type\": \"\",\n                                      \"description\": \"\"\n                                  }\n                              }\n                          }\n                      }\n                  }\n              }\n          ],\n          \"type\": \"dataframe\"}

```

3.0.1 Graphs Of Derived Healthcare Capacity Metrics

```
from plotly.subplots import make_subplots
```

```
# 1. Daily Total System Load
```

```

fig_daily = px.line(
    df,
    x=df.index,
    y='Total System Load',
    title='Daily Total System Load (CBP + HHS)',
    labels={'Total System Load': 'Total Children Under Care', 'index':
'Date'}
)
fig_daily.update_layout(template='plotly_white', hovermode='x
unified')
fig_daily.show()
display(HTML(fig_daily.to_html()))

# 2. Daily Net Daily Intake
fig_net_intake = px.line(
    df,
    x=df.index,
    y='Net Daily Intake',
    title='Daily Net Daily Intake',
    labels={'Net Daily Intake': 'Net Children Intake', 'index':
'Date'}
)
fig_net_intake.update_layout(template='plotly_white', hovermode='x
unified')
fig_net_intake.show()

# 3. Daily Care Load Growth Rate
fig_growth_rate = px.line(
    df,
    x=df.index,
    y='Care Load Growth Rate',
    title='Daily Care Load Growth Rate (%)',
    labels={'Care Load Growth Rate': 'Growth Rate (%)', 'index':
'Date'}
)
fig_growth_rate.update_layout(template='plotly_white', hovermode='x
unified')
fig_growth_rate.show()

```

3.1 3D Surface Plot: Total System Load as a Function of Net Intake and Growth Rate

This 3D surface plot visualizes **Total System Load** as a function of **Net Daily Intake** and **Care Load Growth Rate**. The height of the surface represents the **Total System Load**, allowing for a quick understanding of how different combinations of net intake and growth rate impact the overall system load. This type of plot acts as a 3D heatmap, where the color and height signify the **Total System Load**.

```

import plotly.graph_objects as go
from scipy.interpolate import griddata
import numpy as np
import pandas as pd

# Ensure the necessary columns exist and handle NaNs for this specific
# plot
required_cols_kpi_surface_3d = ['Net Daily Intake', 'Care Load Growth
Rate', 'Total System Load']
df_kpi_surface_3d =
df.dropna(subset=required_cols_kpi_surface_3d).copy()

# Replace any infinite values in 'Care Load Growth Rate' with NaN,
# then drop rows
df_kpi_surface_3d['Care Load Growth Rate'] = df_kpi_surface_3d['Care
Load Growth Rate'].replace([np.inf, -np.inf], np.nan)
df_kpi_surface_3d.dropna(subset=['Care Load Growth Rate'],
inplace=True)

# --- Prepare data for surface plot ---
x_coords_kpi = df_kpi_surface_3d['Net Daily Intake'].values
y_coords_kpi = df_kpi_surface_3d['Care Load Growth Rate'].values
z_values_kpi = df_kpi_surface_3d['Total System Load'].values

# Create a grid for interpolation
x_kpi_finite = x_coords_kpi[np.isfinite(x_coords_kpi)]
y_kpi_finite = y_coords_kpi[np.isfinite(y_coords_kpi)]

grid_x_kpi, grid_y_kpi = np.mgrid[
    x_kpi_finite.min():x_kpi_finite.max():100j,
    y_kpi_finite.min():y_kpi_finite.max():100j
]

# Interpolate the z_values (Total System Load) onto the grid
grid_z_kpi = griddata(
    (x_coords_kpi, y_coords_kpi),
    z_values_kpi,
    (grid_x_kpi, grid_y_kpi),
    method='cubic'
)

# Create the 3D surface plot for KPIs
fig_kpi_surface_3d = go.Figure(data=[
    go.Surface(
        z=grid_z_kpi,
        x=grid_x_kpi,
        y=grid_y_kpi,
        colorscale='Viridis', # Use a distinct colorscale
        colorbar=dict(title='<b>Total System Load</b>', x=1.0),
        cmin=z_values_kpi.min(),

```

```

        cmap=z_values_kpi.max()
    )
])

fig_kpi_surface_3d.update_layout(
    title=dict(
        text='<b>3D Surface Plot: Total System Load vs. Net Intake &
Growth Rate</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Net Daily Intake',
        yaxis_title='Care Load Growth Rate (%)',
        zaxis_title='Total System Load'
    )
)

fig_kpi_surface_3d.show()

```

3.2 3D Line Plot: KPI Evolution Over Time

This 3D line plot illustrates the trajectory of the system's state over time, showing how **Net Daily Intake**, **Care Load Growth Rate**, and **Total System Load** evolve simultaneously. Each point on the line represents a day, and the progression of the line reveals the dynamic interplay of these crucial KPIs.

```

import plotly.graph_objects as go
import numpy as np
import pandas as pd

# Ensure the necessary columns exist and handle NaNs for this specific
plot
required_cols_kpi_line_3d = ['Net Daily Intake', 'Care Load Growth
Rate', 'Total System Load']
df_kpi_line_3d = df.dropna(subset=required_cols_kpi_line_3d).copy()

# Replace any infinite values in 'Care Load Growth Rate' with NaN,
then drop rows
df_kpi_line_3d['Care Load Growth Rate'] = df_kpi_line_3d['Care Load
Growth Rate'].replace([np.inf, -np.inf], np.nan)
df_kpi_line_3d.dropna(subset=['Care Load Growth Rate'], inplace=True)

# Convert datetime index to numerical representation (e.g., epoch
timestamps) for color scaling
df_kpi_line_3d['Time_Numeric'] = df_kpi_line_3d.index.astype(int) /
10**9 # Convert to Unix timestamp in seconds

```

```

fig_kpi_line_3d = go.Figure(data=[
    go.Scatter3d(
        x=df_kpi_line_3d['Net Daily Intake'],
        y=df_kpi_line_3d['Care Load Growth Rate'],
        z=df_kpi_line_3d['Total System Load'],
        mode='lines+markers', # Show both lines and markers for
clearer trajectory
        marker=dict(
            size=3,
            color=df_kpi_line_3d['Time_Numeric'], # Color by numerical
time
            colorscale='Plasma', # Choose an appropriate
colorscale for time
            opacity=0.8,
            colorbar=dict(title='<b>Time (Unix Timestamp)</b>', x=1.0)
# Add color bar for time
        ),
        line=dict(
            color='blue',
            width=2
        )
    )
])

# Update layout for better readability
fig_kpi_line_3d.update_layout(
    scene=dict(
        xaxis_title='Net Daily Intake',
        yaxis_title='Care Load Growth Rate (%)',
        zaxis_title='Total System Load'
    ),
    title=dict(
        text='<b>3D Line Plot: KPI Evolution Over Time</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    )
)

fig_kpi_line_3d.show()

```

4. Trend & Temporal Analysis

```

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Daily Total System Load
fig_daily = px.line( df, x=df.index, y='Total System Load',

```

```

title='Daily Total System Load (CBP + HHS)',
      labels={'Total System Load': 'Total Children Under
Care', 'index': 'Date'} )
fig_daily.update_layout(template='plotly_white', hovermode='x
unified')
fig_daily.show()

# Weekly and Monthly care load trends
weekly_load = df['Total System Load'].resample('W').sum()
monthly_load = df['Total System Load'].resample('ME').sum()

fig_trends = make_subplots(rows=2, cols=1, shared_xaxes=True,
subplot_titles=('Weekly Total System Load', 'Monthly Total System
Load'))

fig_trends.add_trace( go.Scatter(x=weekly_load.index,
y=weekly_load.values, mode='lines', name='Weekly Load'), row=1,
col=1 )

fig_trends.add_trace( go.Scatter(x=monthly_load.index,
y=monthly_load.values, mode='lines', name='Monthly Load'), row=2,
col=1 )

fig_trends.update_layout( title_text="Weekly and Monthly Care Load
Trends", template='plotly_white', hovermode='x unified' )
fig_trends.show()

# Identification of sustained high-load periods
threshold = df['Total System Load'].quantile(0.90) # Top 10% as high
load
high_load_periods = df[df['Total System Load'] > threshold]

fig_highload = px.line( df, x=df.index, y='Total System Load',
title=f'Total System Load with High-Load Threshold ({threshold:.0f})',
      labels={'Total System Load': 'Total Children Under Care', 'index':
'Date'} )

# Add threshold line
fig_highload.add_hline(y=threshold, line_dash="dash",
line_color="red", annotation_text="90th Percentile Threshold")

# Highlight high-load points
fig_highload.add_trace(
    go.Scatter( x=high_load_periods.index, y=high_load_periods['Total
System Load'], mode='markers', marker=dict(color='red', size=8),
name='High Load Periods')
)

fig_highload.update_layout(template='plotly_white', hovermode='x
unified')

```

```
fig_highload.show()

print(f"\nDates with Total System Load above 90th percentile
({threshold:.0f}):")
display(high_load_periods['Total System Load'].head())
```

Dates with Total System Load above 90th percentile (9089):

```
2023-08-10    9149.0
2023-08-16    9879.0
2023-08-17    9913.0
2023-08-20    9693.0
2023-08-21    9871.0
Name: Total System Load, dtype: float64
```

4.1 Monthly Load Heatmap

```
import plotly.express as px
import plotly.graph_objects as go

# Ensure datetime index
if not isinstance(df.index, pd.DatetimeIndex):
    df.index = pd.to_datetime(df.index)

# Prepare monthly load data
monthly_load_heatmap_data = df['Total System
Load'].resample('ME').sum().to_frame()
monthly_load_heatmap_data['Year'] =
monthly_load_heatmap_data.index.year
monthly_load_heatmap_data['Month'] =
monthly_load_heatmap_data.index.month_name()

# Pivot for heatmap
month_order = [
    'January', 'February', 'March', 'April', 'May', 'June',
    'July', 'August', 'September', 'October', 'November', 'December'
]
monthly_load_pivot = monthly_load_heatmap_data.pivot(index='Year',
columns='Month', values='Total System Load')
monthly_load_pivot = monthly_load_pivot[month_order]

# Plotly heatmap
fig = go.Figure(data=go.Heatmap( z=monthly_load_pivot.values,
x=monthly_load_pivot.columns, y=monthly_load_pivot.index,
colorscale='Viridis',
text=monthly_load_pivot.values,
texttemplate="%{text:,.0f}", # formatted numbers with commas
hovertemplate="Year %{y}, %{x}: %{z:,.0f}<extra></extra>"
))
```

```

fig.update_layout(
    title="Monthly Total System Load Heatmap",
    xaxis_title="Month",
    yaxis_title="Year",
    template="plotly_white"
)

fig.show()

```

4.2 Discharge Effectiveness Over Time

```

import plotly.express as px
import plotly.graph_objects as go

# Calculate Discharge Offset Ratio
df['Discharge Offset Ratio'] = df['Children discharged from HHS Care']
/ df['Children transferred out of CBP custody']
df['Discharge Offset Ratio'] = df['Discharge Offset
Ratio'].replace([float('inf'), -float('inf')], 0).fillna(0)
avg_discharge_offset = df['Discharge Offset Ratio'].mean()

# Interactive line chart
fig = px.line(
    df,
    x=df.index,
    y='Discharge Offset Ratio',
    title='<b>Discharge Effectiveness (Discharge Offset Ratio) Over
Time</b>',
    labels={'Discharge Offset Ratio': 'Discharge Offset Ratio',
'index': 'Date'}
)

# Add average line
fig.add_hline(
    y=avg_discharge_offset,
    line_dash="dash",
    line_color="red",
    annotation_text=f"Average: {avg_discharge_offset:.2f}",
    annotation_position="top left"
)

fig.update_layout(
    template='plotly_white',
    xaxis_title='Date',
    yaxis_title='Discharge Offset Ratio',
    hovermode='x unified',
    title=dict(x=0.5, xanchor='center') # Set title properties using
update_layout
)

```



```
fig.show()
```

4.3 Interactive Flow Efficiency Analysis

We use a 2D marginal plot to visualize the distribution of 'Net Daily Intake' against the 'Discharge Offset Ratio', identifying high-density operational states.

```
import plotly.express as px

# Joint Distribution of Inflow vs Outflow Efficiency using Scatter
# with Marginals
fig_flow_dist = px.scatter(
    df.dropna(subset=['Net Daily Intake', 'Discharge Offset Ratio']),
    x='Net Daily Intake',
    y='Discharge Offset Ratio',
    marginal_x='histogram',
    marginal_y='histogram',
    trendline='ols',
    title='<b>Joint Distribution: Net Intake vs. Discharge
Efficiency',
    labels={'Net Daily Intake': '<b>Net Inflow Count', 'Discharge
Offset Ratio': '<b>Outflow Ratio'},
    template='plotly_white'
)
fig_flow_dist.show()
```

4.4 3D Trends

a. 3D Line Plot: Total System Load and Rolling Average Over Time

This 3D line plot illustrates the progression of 'Total System Load' and its '7-Day Rolling Avg Load' over time, with 'Net Daily Intake' as the third dimension. The color of the line represents the time progression, providing a visual cue for how these metrics evolve together.

```
import plotly.graph_objects as go
import numpy as np
import pandas as pd

df['7-Day Rolling Avg Load'] = df['Total System
Load'].rolling(window=7).mean()

# Ensure the necessary columns exist and handle NaNs for this specific
# plot
required_cols_trend_line_3d = ['Total System Load', '7-Day Rolling Avg
Load', 'Net Daily Intake']
df_trend_line_3d =
```

```

df.dropna(subset=required_cols_trend_line_3d).copy()

# Convert datetime index to numerical representation (e.g., epoch
# timestamps) for color scaling
df_trend_line_3d['Time_Numeric'] = df_trend_line_3d.index.astype(int)
/ 10**9 # Convert to Unix timestamp in seconds

fig_trend_line_3d = go.Figure(data=[
    go.Scatter3d(
        x=df_trend_line_3d['Total System Load'],
        y=df_trend_line_3d['7-Day Rolling Avg Load'],
        z=df_trend_line_3d['Net Daily Intake'],
        mode='lines+markers', # Show both lines and markers for
# clearer trajectory
        marker=dict(
            size=3,
            color=df_trend_line_3d['Time_Numeric'], # Color by
# numerical time
            colorscale='Viridis', # Choose an appropriate
# colorscale for time
            opacity=0.8,
            colorbar=dict(title='<b>Time (Unix Timestamp)</b>', x=1.0)
# Add color bar for time
        ),
        line=dict(
            color='purple',
            width=2
        )
    )
])

# Update layout for better readability
fig_trend_line_3d.update_layout(
    scene=dict(
        xaxis_title='Total System Load',
        yaxis_title='7-Day Rolling Avg Load',
        zaxis_title='Net Daily Intake'
    ),
    title=dict(
        text='<b>3D Line Plot: Total System Load, Rolling Avg, and Net
Intake Over Time</b>',
        x=0.5, # Center the title horizontally
        xanchor='center',
        font=dict(size=17) # Increase font size
    )
)

fig_trend_line_3d.show()

```

b. 3D Surface Plot: Total System Load vs. Day of Week and Month

This 3D surface plot visualizes the 'Total System Load' as a function of the 'Day_of_Week' and 'Month'. The height of the surface represents the 'Total System Load', allowing for insights into potential weekly or monthly seasonal patterns. This acts as a 3D heatmap, with color and height indicating the system load.

```
import plotly.graph_objects as go
from scipy.interpolate import griddata
import numpy as np
import pandas as pd

# Ensure 'Day_of_Week' and 'Month' features are available in df
# If not already present from feature engineering, create them
if 'Day_of_Week' not in df.columns:
    df['Day_of_Week'] = df.index.dayofweek
if 'Month' not in df.columns:
    df['Month'] = df.index.month

# Ensure the necessary columns exist and handle NaNs for this specific plot
required_cols_temporal_surface_3d = ['Total System Load',
                                       'Day_of_Week', 'Month']
df_temporal_surface_3d =
df.dropna(subset=required_cols_temporal_surface_3d).copy()

# --- Prepare data for surface plot ---
x_coords_temporal = df_temporal_surface_3d['Day_of_Week'].values
y_coords_temporal = df_temporal_surface_3d['Month'].values
z_values_temporal = df_temporal_surface_3d['Total System Load'].values

# Create a grid for interpolation
x_temporal_finite = x_coords_temporal[np.isfinite(x_coords_temporal)]
y_temporal_finite = y_coords_temporal[np.isfinite(y_coords_temporal)]

grid_x_temporal, grid_y_temporal = np.mgrid[
    x_temporal_finite.min():x_temporal_finite.max():10j, # Fewer
    y_temporal_finite.min():y_temporal_finite.max():12j # 12 months
    points for categories
]

# Interpolate the z_values (Total System Load) onto the grid
grid_z_temporal = griddata(
    (x_coords_temporal, y_coords_temporal),
    z_values_temporal,
    (grid_x_temporal, grid_y_temporal),
    method='linear' # Use linear for less interpolation on
    categorical-like data
)
```

```

# Create the 3D surface plot for temporal patterns
fig_temporal_surface_3d = go.Figure(data=[
    go.Surface(
        z=grid_z_temporal,
        x=grid_x_temporal,
        y=grid_y_temporal,
        colorscale='Plasma', # Use a distinct colorscale
        colorbar=dict(title='<b>Total System Load</b>', x=1.0),
        cmin=z_values_temporal.min(),
        cmx=z_values_temporal.max()
    )
])

fig_temporal_surface_3d.update_layout(
    title=dict(
        text='<b>3D Surface Plot: Total System Load vs. Day of Week
and Month</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Day of Week (0=Mon, 6=Sun)',
        yaxis_title='Month (1=Jan, 12=Dec)',
        zaxis_title='Total System Load'
    )
)

fig_temporal_surface_3d.show()

```

5. Pressure & Stress Identification

Plotly Visualizations for Pressure & Stress Identification

```

import plotly.graph_objects as go
import plotly.express as px

# Rolling averages (7-day, 14-day) for Total System Load
df['7-Day Rolling Avg Load'] = df['Total System
Load'].rolling(window=7).mean()
df['14-Day Rolling Avg Load'] = df['Total System
Load'].rolling(window=14).mean()

# Interactive line chart for Total System Load with rolling averages
fig1 = go.Figure()
fig1.add_trace(go.Scatter(x=df.index, y=df['Total System Load'],
                        mode='lines', name='Daily Load',
                        line=dict(color='blue'), opacity=0.6))
fig1.add_trace(go.Scatter(x=df.index, y=df['7-Day Rolling Avg Load'],

```

```

mode='lines', name='7-Day Rolling Avg',
line=dict(color='orange'))
fig1.add_trace(go.Scatter(x=df.index, y=df['14-Day Rolling Avg Load'],
mode='lines', name='14-Day Rolling Avg',
line=dict(color='green'))))
fig1.update_layout(title=dict(text='<b>Total System Load with Rolling
Averages</b>', x=0.5, xanchor='center'),
xaxis_title='Date', yaxis_title='Total Children
Under Care',
template='plotly_white')
fig1.show()

# Detection of prolonged strain windows
df['Sustained Positive Net Intake'] = (df['Net Daily Intake'] >
0).rolling(window=3).apply(lambda x: x.all(), raw=True).fillna(0)
mean_load = df['Total System Load'].mean()
df['High Load'] = (df['Total System Load'] > mean_load).astype(int)

strain_windows = df[(df['Sustained Positive Net Intake'] == 1) &
(df['High Load'] == 1)].copy()

if not strain_windows.empty:
    print("\nDetected prolonged strain windows (3+ days positive net
intake AND above average total load):\n")
    strain_windows['group'] =
(strain_windows.index.to_series().diff().dt.days > 1).cumsum()
    for _, group_df in strain_windows.groupby('group'):
        print(f"    Start: {group_df.index.min().strftime('%Y-%m-%d')},
End: {group_df.index.max().strftime('%Y-%m-%d')}")

    # Highlight strain windows on the load chart
    fig2 = go.Figure()
    fig2.add_trace(go.Scatter(x=df.index, y=df['Total System Load'],
mode='lines', name='Daily Load',
line=dict(color='blue'))))
    fig2.add_trace(go.Scatter(x=strain_windows.index,
y=strain_windows['Total System Load'],
mode='markers', name='Strain Window',
marker=dict(color='red', size=8)))
    fig2.update_layout(title=dict(text='<b>Detected Strain Windows on
Total System Load</b>', x=0.5, xanchor='center'),
xaxis_title='Date', yaxis_title='Total Children
Under Care',
template='plotly_white')
    fig2.show()
else:
    print("\nNo prolonged strain windows detected under the defined
conditions.")

```

```
print("\nSample:\n")
display(df.tail())
```

Detected prolonged strain windows (3+ days positive net intake AND above average total load):

Start:	2024-02-07,	End:	2024-02-08
Start:	2024-02-14,	End:	2024-02-14
Start:	2024-02-21,	End:	2024-02-21
Start:	2024-02-29,	End:	2024-02-29
Start:	2024-03-06,	End:	2024-03-07
Start:	2024-04-23,	End:	2024-04-25
Start:	2024-04-30,	End:	2024-05-02
Start:	2024-05-08,	End:	2024-05-09
Start:	2024-05-14,	End:	2024-05-15
Start:	2024-05-22,	End:	2024-05-23
Start:	2024-05-29,	End:	2024-05-30
Start:	2024-06-12,	End:	2024-06-12
Start:	2024-06-19,	End:	2024-06-20
Start:	2024-07-09,	End:	2024-07-10
Start:	2024-07-31,	End:	2024-08-01
Start:	2024-08-08,	End:	2024-08-08
Start:	2024-08-14,	End:	2024-08-14
Start:	2024-08-20,	End:	2024-08-22
Start:	2024-08-29,	End:	2024-08-29
Start:	2024-09-04,	End:	2024-09-05
Start:	2024-09-11,	End:	2024-09-11
Start:	2024-09-25,	End:	2024-09-25
Start:	2024-10-02,	End:	2024-10-02
Start:	2024-10-09,	End:	2024-10-10
Start:	2024-10-16,	End:	2024-10-17
Start:	2024-10-22,	End:	2024-10-23
Start:	2024-10-30,	End:	2024-10-30
Start:	2024-11-06,	End:	2024-11-07
Start:	2024-11-13,	End:	2024-11-14
Start:	2024-11-20,	End:	2024-11-20
Start:	2024-11-27,	End:	2024-11-28
Start:	2024-12-03,	End:	2024-12-05
Start:	2024-12-12,	End:	2024-12-12
Start:	2024-12-18,	End:	2024-12-18
Start:	2024-12-26,	End:	2024-12-26

Sample:

```
{"summary":{"\n  \"name\": \"display(df\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"Children apprehended and\nplaced in CBP custody*\", \n      \"properties\": {\n        \"dtype\":
```

```

\"number\", \n          \"std\": 4.764451699828639, \n          \"min\": 0.0, \n          \"max\": 11.0, \n          \"num_unique_values\": 4, \n          \"samples\": [\n            11.0, \n            6.0, \n            7.0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n        {\n          \"column\": \"Children in CBP custody\", \n          \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 21.358838919754042, \n            \"min\": 0.0, \n            \"max\": 50.0, \n            \"num_unique_values\": 4, \n            \"samples\": [\n              50.0, \n              18.0, \n              31.0 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n          }, \n          {\n            \"column\": \"Children transferred out of CBP custody\", \n            \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 5.504543577809153, \n              \"min\": 0.0, \n              \"max\": 11.0, \n              \"num_unique_values\": 3, \n              \"samples\": [\n                11.0, \n                6.0, \n                0.0 \n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\" \n            }, \n            {\n              \"column\": \"Children in HHS Care\", \n              \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 1357.8114007475413, \n                \"min\": 0.0, \n                \"max\": 2484.0, \n                \"num_unique_values\": 4, \n                \"samples\": [\n                  2472.0, \n                  2484.0, \n                  2481.0 \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\" \n              }, \n              {\n                \"column\": \"Children discharged from HHS Care\", \n                \"properties\": {\n                  \"dtype\": \"number\", \n                  \"std\": 7.615773105863909, \n                  \"min\": 0.0, \n                  \"max\": 16.0, \n                  \"num_unique_values\": 4, \n                  \"samples\": [\n                    16.0, \n                    14.0, \n                    10.0 \n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\" \n                }, \n                {\n                  \"column\": \"Total System Load\", \n                  \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 1375.8972345346144, \n                    \"min\": 0.0, \n                    \"max\": 2522.0, \n                    \"num_unique_values\": 4, \n                    \"samples\": [\n                      2522.0, \n                      2502.0, \n                      2512.0 \n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\" \n                  }, \n                  {\n                    \"column\": \"Net Daily Intake\", \n                    \"properties\": {\n                      \"dtype\": \"number\", \n                      \"std\": 4.505552130427524, \n                      \"min\": -10.0, \n                      \"max\": 1.0, \n                      \"num_unique_values\": 4, \n                      \"samples\": [\n                        -10.0, \n                        -3.0, \n                        1.0 \n                      ], \n                      \"semantic_type\": \"\", \n                      \"description\": \"\" \n                    }, \n                    {\n                      \"column\": \"Care Load Growth Rate\", \n                      \"properties\": {\n                        \"dtype\": \"number\", \n                        \"std\": 57.736849553139926, \n                        \"min\": -100.0, \n                        \"max\": 0.398089171974525, \n                        \"num_unique_values\": 3, \n                        \"samples\": [\n                          -0.3965107057890549, \n                          0.398089171974525, \n                          -100.0 \n                        ], \n                        \"semantic_type\": \"\", \n                        \"description\": \"\" \n                      }, \n                      {\n                        \"column\": \"Positive Net Intake\", \n                        \"properties\": {\n                          \"dtype\": \"number\", \n                          \"std\": 0, \n                          \"min\": 0, \n                          \"max\": 1, \n                        }, \n                      }, \n                    }, \n                  }, \n                }, \n              }, \n            }, \n          }, \n        }, \n      ], \n    }, \n  ], \n}

```

```

{"num_unique_values": 2, "samples": [0, 1], "semantic_type": "", "description": "Discharge Offset Ratio", "column": "Discharge Offset Ratio", "properties": {"dtype": "number", "std": 1.1017975128907784, "min": 0.0, "max": 2.6666666666666665, "num_unique_values": 4, "samples": [2.6666666666666665, 1.2727272727272727], "semantic_type": "", "description": "7-Day Rolling Avg Load", "column": "7-Day Rolling Avg Load", "properties": {"dtype": "number", "std": 2.526593253649584, "min": 1789.5714285714287, "max": 1795.7142857142858, "num_unique_values": 3, "samples": [1789.5714285714287, 1795.0]}, "semantic_type": "", "description": "Day of Week", "column": "Day of Week", "properties": {"dtype": "int32", "num_unique_values": 5, "samples": [6, 3], "semantic_type": "", "description": "Month", "column": "Month", "properties": {"dtype": "int32", "num_unique_values": 1, "samples": [12], "semantic_type": "", "description": "14-Day Rolling Avg Load", "column": "14-Day Rolling Avg Load", "properties": {"dtype": "number", "std": 2.8958978728187827, "min": 1776.4285714285713, "max": 1784.5714285714287, "num_unique_values": 3, "samples": [1776.4285714285713]}, "semantic_type": "", "description": "Sustained Positive Net Intake", "column": "Sustained Positive Net Intake", "properties": {"dtype": "number", "std": 0.44721359549995804, "min": 0.0, "max": 1.0, "num_unique_values": 2, "samples": [0.0]}, "semantic_type": "", "description": "High Load", "column": "High Load", "properties": {"dtype": "number", "std": 0, "min": 0, "max": 0, "num_unique_values": 1, "samples": [0]}, "semantic_type": "", "description": "", "column": ""}], "type": "dataframe"}

```

5.1 3D Scatter Plots for System Load, Variability (7-Day Rolling Std Dev Load) & Strain Window/Net Intake

```

import plotly.graph_objects as go
import pandas as pd

# Ensure the dataframe index is named 'Date' for consistent plotting
if df.index.name != 'Date':

```



```

df.index.name = 'Date'

# Calculate '7-Day Rolling Std Dev Load'
df['7-Day Rolling Std Dev Load'] = df['Total System
Load'].rolling(window=7).std()

# Create a 3D scatter plot of 'Total System Load', '7-Day Rolling Std
Dev Load', and 'Net Daily Intake'
fig = go.Figure(data=[
    go.Scatter3d(
        x=df['Total System Load'],
        y=df['7-Day Rolling Std Dev Load'],
        z=df['Net Daily Intake'],
        mode='markers',
        marker=dict(
            size=5,
            color=df['Total System Load'], # Color by Total System
Load
            colorscale='Viridis', # Choose a colorscale
            opacity=0.8,
            colorbar=dict(title='<b>Total System Load</b>', x=1.0) #
Add color bar
        )
    ])

# Update layout for better readability
fig.update_layout(
    scene=dict(
        xaxis_title='Total System Load',
        yaxis_title='7-Day Rolling Std Dev Load',
        zaxis_title='Net Daily Intake'
    ),
    title=dict(
        text='<b>3D Scatter Plot: System Load, Variability, and Net
Intake</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    )
)

fig.show()

# Another 3D plot focusing on 'Sustained Positive Net Intake' and
'High Load'
# For this, we'll use a more categorical approach or color coding
fig2 = go.Figure(data=[
    go.Scatter3d(
        x=df['Total System Load'],

```

```

        y=df['7-Day Rolling Std Dev Load'],
        z=df['High Load'], # Use High Load as a binary z-axis for
context
        mode='markers',
        marker=dict(
            size=5,
            color=df['Sustained Positive Net Intake'], # Color by
Sustained Positive Net Intake
            colorscale='Plasma', # Different
colorscale
            opacity=0.8,
            colorbar=dict(title='<b>Sustained Positive Net
Intake</b>', x=1.0) # Add color bar
        )
    )
])

# Update layout
fig2.update_layout(
    scene=dict(
        xaxis_title='Total System Load',
        yaxis_title='7-Day Rolling Std Dev Load',
        zaxis_title='High Load (0=No, 1=Yes)'
    ),
    title=dict(
        text='<b>3D Scatter Plot: System Load, Variability, and Strain
Windows</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    )
)

fig2.show()

```

5.2 3D Heatmap Surface Plot for System Stress

To create a 3D heatmap surface plot, we first define a **Composite Stress Score** that combines key stress indicators:

1. **Base Stress:** Calculated as the product of **Total System Load** and **7-Day Rolling Std Dev Load**. This captures periods of high load *and* high variability.
2. **Strain Multiplier:** The score is then multiplied by **(1 + Sustained Positive Net Intake + High Load)**. This amplifies the stress score when either of these binary indicators (prolonged positive net intake or above-average load) is present.
3. **Net Intake Contribution:** Finally, a **Normalized_Net_Intake** (scaled to be between 0 and 1, where 0 represents the minimum net intake and 1 the maximum) is added to the multiplier **(1 + Normalized_Net_Intake)**. This ensures that periods of high net

intake further increase the stress score, while periods of negative net intake reduce it, providing a more nuanced view of accumulating pressure.

After calculating this `Stress_Score` for each data point, `scipy.interpolate.griddata` is used to interpolate these scattered stress values onto a regular 2D grid of 'Total System Load' and '7-Day Rolling Std Dev Load'. This interpolated grid then forms the 'surface' of the 3D plot, with the color representing the intensity of the Composite Stress Score.

```
import plotly.graph_objects as go
from scipy.interpolate import griddata
import numpy as np
import pandas as pd

# Ensure the necessary columns exist after previous steps and handle NaNs
required_cols = ['Total System Load', '7-Day Rolling Std Dev Load',
                 'Net Daily Intake', 'Sustained Positive Net Intake', 'High Load']
df_plot = df.dropna(subset=required_cols).copy()

# Calculate a composite Stress_Score
# 1. Base Stress: Product of Load and Variability
df_plot['Stress_Score'] = df_plot['Total System Load'] * df_plot['7-
Day Rolling Std Dev Load']

# 2. Strain Multiplier: Amplify stress based on binary indicators
# Add 1 to binary indicators so they act as multipliers >= 1
df_plot['Stress_Score'] *= (1 + df_plot['Sustained Positive Net
Intake'] + df_plot['High Load'])

# 3. Net Intake Contribution: Incorporate normalized net daily intake
# Normalize Net Daily Intake to a [0, 1] range to contribute
meaningfully without dominating
min_net_intake = df_plot['Net Daily Intake'].min()
max_net_intake = df_plot['Net Daily Intake'].max()
# Handle case where max_net_intake == min_net_intake to avoid division
by zero
if (max_net_intake - min_net_intake) == 0:
    df_plot['Normalized_Net_Intake'] = 0.5 # Default to mid-range if
no variation
else:
    df_plot['Normalized_Net_Intake'] = (df_plot['Net Daily Intake'] -
min_net_intake) / (max_net_intake - min_net_intake)

df_plot['Stress_Score'] *= (1 + df_plot['Normalized_Net_Intake']) #
Further multiply by a factor influenced by net intake

# --- Prepare data for surface plot ---
x_coords = df_plot['Total System Load'].values
y_coords = df_plot['7-Day Rolling Std Dev Load'].values
z_values = df_plot['Stress_Score'].values
```

```

# Create a grid for interpolation
# Use a denser grid (e.g., 200j for 200 points) for smoother surface
grid_x, grid_y = np.mgrid[x_coords.min():x_coords.max():100j,
y_coords.min():y_coords.max():100j]

# Interpolate the z_values (Stress_Score) onto the grid
# Use 'cubic' method for a smoother surface, 'linear' or 'nearest' are
alternatives
grid_z = griddata(
    (x_coords, y_coords),
    z_values,
    (grid_x, grid_y),
    method='cubic'
)

# Create the 3D surface plot
fig_stress_heatmap_3d = go.Figure(data=[
    go.Surface(
        z=grid_z,
        x=grid_x,
        y=grid_y,
        colorscale='Hot', # 'Hot' colorscale typically used for
heatmaps
        colorbar=dict(title='<b>Composite Stress Score</b>', x=1.0),
        cmin=z_values.min(), # Set color range based on actual stress
values
        cmap=z_values.max()
    )
])

fig_stress_heatmap_3d.update_layout(
    title=dict(
        text='<b>3D Heatmap Surface Plot of System Stress</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Total System Load',
        yaxis_title='7-Day Rolling Std Dev Load',
        zaxis_title='Composite Stress Score'
    )
)

fig_stress_heatmap_3d.show()

```

5.3 3D Heatmap Surface Plot for System Pressure

To complement the stress analysis, we create a 3D heatmap surface plot to visualize **System Pressure**. The **Composite Pressure Score** is derived from:

1. **Base Pressure**: Directly from the **Net Daily Intake**, normalized to a [0, 1] range to ensure proportional contribution.
2. **Growth Rate Multiplier**: A positive **Care Load Growth Rate** (day-over-day percentage change in total system load) amplifies the pressure. Only positive growth rates contribute to this multiplier, reflecting increasing inbound pressure.
3. **Sustained Intake Multiplier**: The score is amplified if there is **Sustained Positive Net Intake** (indicating prolonged periods where more children enter care than leave).
4. **High Load Multiplier**: The score is further amplified if the system is already under **High Load** (total system load above average).

These components are combined to create a **Pressure_Score** for each data point. Similar to the stress plot, `scipy.interpolate.griddata` is then used to interpolate these scattered pressure values onto a regular 2D grid, visualizing the intensity of pressure across different combinations of 'Net Daily Intake' and 'Care Load Growth Rate'.

```
import plotly.graph_objects as go
from scipy.interpolate import griddata
import numpy as np
import pandas as pd

# Ensure the necessary columns exist after previous steps and handle NaNs
required_pressure_cols = ['Net Daily Intake', 'Care Load Growth Rate',
                           'Sustained Positive Net Intake', 'High Load']
df_plot_pressure = df.dropna(subset=required_pressure_cols).copy()

# --- Calculate a composite Pressure_Score ---

# 1. Normalize Net Daily Intake
min_net_intake = df_plot_pressure['Net Daily Intake'].min()
max_net_intake = df_plot_pressure['Net Daily Intake'].max()

# Handle case where max_net_intake == min_net_intake to avoid division by zero
if (max_net_intake - min_net_intake) == 0:
    df_plot_pressure['Normalized_Net_Daily_Intake'] = 0.5 # Default to mid-range if no variation
else:
    df_plot_pressure['Normalized_Net_Daily_Intake'] =
    (df_plot_pressure['Net Daily Intake'] - min_net_intake) /
    (max_net_intake - min_net_intake)

# Base Pressure Score
df_plot_pressure['Pressure_Score'] =
df_plot_pressure['Normalized_Net_Daily_Intake']
```

```

# 2. Growth Rate Multiplier (only positive growth contributes to
pressure)
# Ensure 'Care Load Growth Rate' is not inf or nan before using
df_plot_pressure['Care Load Growth Rate'] = df_plot_pressure['Care
Load Growth Rate'].replace([np.inf, -np.inf], np.nan).fillna(0)

# Create a positive growth rate, and scale it down to a reasonable
multiplier (e.g., divide by 100 for percentage)
df_plot_pressure['Positive_Growth_Rate_Multiplier'] =
(df_plot_pressure['Care Load Growth Rate'].apply(lambda x: max(0, x))
/ 100) # Assuming rate is percentage

df_plot_pressure['Pressure_Score'] *= (1 +
df_plot_pressure['Positive_Growth_Rate_Multiplier'])

# 3. Sustained Intake Multiplier
df_plot_pressure['Pressure_Score'] *= (1 + df_plot_pressure['Sustained
Positive Net Intake'])

# 4. High Load Multiplier
df_plot_pressure['Pressure_Score'] *= (1 + df_plot_pressure['High
Load'])

# --- Prepare data for surface plot ---
x_coords_pressure = df_plot_pressure['Net Daily Intake'].values
y_coords_pressure = df_plot_pressure['Care Load Growth Rate'].values
z_values_pressure = df_plot_pressure['Pressure_Score'].values

# Create a grid for interpolation
# Use a denser grid for smoother surface, and ensure it covers the
range of x and y coords
# Filter out potential inf values from x_coords and y_coords before
min/max
x_coords_pressure_finite =
x_coords_pressure[np.isfinite(x_coords_pressure)]
y_coords_pressure_finite =
y_coords_pressure[np.isfinite(y_coords_pressure)]

grid_x_pressure, grid_y_pressure = np.mgrid[
x_coords_pressure_finite.min():x_coords_pressure_finite.max():100j,
y_coords_pressure_finite.min():y_coords_pressure_finite.max():100j
]

# Interpolate the z_values (Pressure_Score) onto the grid
grid_z_pressure = griddata(
(x_coords_pressure, y_coords_pressure),
z_values_pressure,
(grid_x_pressure, grid_y_pressure),

```

```

        method='cubic'
    )

# Create the 3D surface plot for pressure
fig_pressure = go.Figure(data=[
    go.Surface(
        z=grid_z_pressure,
        x=grid_x_pressure,
        y=grid_y_pressure,
        colorscale='Plasma', # Different colorscale for distinction
        colorbar=dict(title='<b>Composite Pressure Score</b>', x=1.0),
        cmin=z_values_pressure.min(),
        cmap=z_values_pressure.max()
    )
])

fig_pressure.update_layout(
    title=dict(
        text='<b>3D Heatmap Surface Plot of System Pressure</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Net Daily Intake',
        yaxis_title='Care Load Growth Rate (%)',
        zaxis_title='Composite Pressure Score'
    )
)

fig_pressure.show()

```

6. Key Performance Indicators (KPIs)

```

# 1. Total Children Under Care (Current and Average)
current_total_care = df['Total System Load'].iloc[-1]
avg_total_care = df['Total System Load'].mean()

# 2. Net Intake Pressure (Current Net Intake)
current_net_intake = df['Net Daily Intake'].iloc[-1]

# 3. Care Load Volatility Index (Standard Deviation of growth rate)
care_load_volatility = df['Care Load Growth Rate'].std()

# 4. Backlog Accumulation Rate (Mean Net Intake over the last 30 days)
backlog_accumulation_rate = df['Net Daily Intake'].tail(30).mean()

# 5. Discharge Offset Ratio (Discharges / Transfers into HHS)
# To avoid division by zero, we handle cases where transfers might be 0

```

```

df['Discharge Offset Ratio'] = df['Children discharged from HHS Care']
/ df['Children transferred out of CBP custody']
df['Discharge Offset Ratio'] = df['Discharge Offset
Ratio'].replace([float('inf'), -float('inf')], 0).fillna(0)
avg_discharge_offset = df['Discharge Offset Ratio'].mean()

print(f"--- KPI Summary ---")
print(f"Total Children Under Care (Current):
{current_total_care:,.0f}")
print(f"Average System-wide Responsibility: {avg_total_care:,.0f}")
print(f"Current Net Intake Pressure: {current_net_intake:,.0f}")
print(f"Care Load Volatility Index: {care_load_volatility:.2f}%")
print(f"Backlog Accumulation Rate (30-day avg):
{backlog_accumulation_rate:.2f}")
print(f"Average Discharge Offset Ratio: {avg_discharge_offset:.2f}")

--- KPI Summary ---
Total Children Under Care (Current): 2,502
Average System-wide Responsibility: 4,175
Current Net Intake Pressure: -3
Care Load Volatility Index: 41.88%
Backlog Accumulation Rate (30-day avg): 0.37
Average Discharge Offset Ratio: 1.34

```

3D Scatter Plot: Net Daily Intake, Discharge Offset Ratio, and Total System Load Over Time

This 3D scatter plot visualizes the dynamic relationship between **Net Daily Intake**, **Discharge Offset Ratio**, and **Total System Load** over time. Each point represents a day, with the color indicating the progression of time, allowing for the identification of patterns and trajectories where these three crucial derived metrics interact.

```

import plotly.graph_objects as go
import numpy as np
import pandas as pd

# Ensure the necessary columns exist and handle NaNs for this specific
plot
required_cols_discharge_scatter = ['Net Daily Intake', 'Discharge
Offset Ratio', 'Total System Load']
df_discharge_scatter_3d =
df.dropna(subset=required_cols_discharge_scatter).copy()

# Convert datetime index to numerical representation (e.g., epoch
timestamps) for color scaling
df_discharge_scatter_3d['Time_Numeric'] =
df_discharge_scatter_3d.index.astype(int) / 10**9 # Convert to Unix
timestamp in seconds

```



```

fig_discharge_scatter_3d = go.Figure(data=[
    go.Scatter3d(
        x=df_discharge_scatter_3d['Net Daily Intake'],
        y=df_discharge_scatter_3d['Discharge Offset Ratio'],
        z=df_discharge_scatter_3d['Total System Load'],
        mode='markers',
        marker=dict(
            size=5,
            color=df_discharge_scatter_3d['Time_Numeric'], # Color by
numerical time
            colorscale='Viridis', # Choose an appropriate
colorscale for time
            opacity=0.8,
            colorbar=dict(title='<b>Time (Unix Timestamp)</b>', x=1.0)
# Add color bar for time
        ),
        text=[
            f'Date: {date.strftime('%Y-%m-%d')}<br>Net Intake:
{ni:.0f}<br>Discharge Ratio: {dor:.2f}<br>Total Load: {tsl:.0f}'
            for date, ni, dor, tsl in zip(
                df_discharge_scatter_3d.index,
                df_discharge_scatter_3d['Net Daily Intake'],
                df_discharge_scatter_3d['Discharge Offset Ratio'],
                df_discharge_scatter_3d['Total System Load']
            )
        ],
        hoverinfo='text'
    )
])

fig_discharge_scatter_3d.update_layout(
    scene=dict(
        xaxis_title='Net Daily Intake',
        yaxis_title='Discharge Offset Ratio',
        zaxis_title='Total System Load'
    ),
    title=dict(
        text='<b>3D Scatter Plot: Net Daily Intake, Discharge Offset
Ratio, and Total System Load Over Time</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    )
)

fig_discharge_scatter_3d.show()

```

3D Surface Plot: Total System Load as a Function of Net Daily Intake and Discharge Offset Ratio

This 3D surface plot visualizes **Total System Load** as a function of **Net Daily Intake** and **Discharge Offset Ratio**. The height and color of the surface represent the **Total System Load**, allowing for a clear understanding of how the interaction of inflow (Net Daily Intake) and outflow efficiency (Discharge Offset Ratio) impacts the overall system burden.

```
import plotly.graph_objects as go
from scipy.interpolate import griddata
import numpy as np
import pandas as pd

# Ensure the necessary columns exist and handle NaNs for this specific plot
required_cols_discharge_surface = ['Net Daily Intake', 'Discharge Offset Ratio', 'Total System Load']
df_discharge_surface_3d =
df.dropna(subset=required_cols_discharge_surface).copy()

# --- Prepare data for surface plot ---
x_coords_ds = df_discharge_surface_3d['Net Daily Intake'].values
y_coords_ds = df_discharge_surface_3d['Discharge Offset Ratio'].values
z_values_ds = df_discharge_surface_3d['Total System Load'].values

# Create a grid for interpolation
x_ds_finite = x_coords_ds[np.isfinite(x_coords_ds)]
y_ds_finite = y_coords_ds[np.isfinite(y_coords_ds)]

grid_x_ds, grid_y_ds = np.mgrid[
    x_ds_finite.min():x_ds_finite.max():100j,
    y_ds_finite.min():y_ds_finite.max():100j
]

# Interpolate the z_values (Total System Load) onto the grid
grid_z_ds = griddata(
    (x_coords_ds, y_coords_ds),
    z_values_ds,
    (grid_x_ds, grid_y_ds),
    method='cubic'
)

# Create the 3D surface plot
fig_discharge_surface_3d = go.Figure(data=[
    go.Surface(
        z=grid_z_ds,
        x=grid_x_ds,
        y=grid_y_ds,
        colorscale='Plasma', # Use a distinct colorscale
    )
])
```

```

        colorbar=dict(title='<b>Total System Load</b>', x=1.0),
        cmin=z_values_ds.min(),
        cmap=z_values_ds.max()
    )
1)

fig_discharge_surface_3d.update_layout(
    title=dict(
        text='<b>3D Surface Plot: Total System Load vs. Net Daily
Intake & Discharge Offset Ratio</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Net Daily Intake',
        yaxis_title='Discharge Offset Ratio',
        zaxis_title='Total System Load'
    )
)

fig_discharge_surface_3d.show()

```

7. Insights and Recommendations

Based on the comprehensive analysis of the HHS Unaccompanied Alien Children Program data, covering data ingestion, quality validation, feature engineering, trend and temporal analysis, pressure & stress identification, and the derivation of Key Performance Indicators (KPIs), several crucial insights and actionable recommendations emerge before moving into advanced modeling.

Key Insights:

- **Data Quality Issues:** Initial data cleaning addressed missing dates and non-numeric entries. However, anomalies such as 'Children transferred out of CBP custody' exceeding 'Children in CBP custody' were identified, suggesting potential reporting discrepancies that require further investigation. The `fillna(0)` strategy for missing data points, while practical for re-indexing, might influence the performance of certain linear models by introducing artificial zero-valued periods.
- **Dynamic System Load & Trends:** The 'Total System Load' (CBP + HHS) exhibits significant daily fluctuations, but clear underlying weekly and monthly patterns are evident. Rolling averages effectively smooth these fluctuations, clarifying trends, while the '7-Day Rolling Std Dev Load' highlights periods of heightened variability.
- **Operational Strain:** The analysis successfully identified "prolonged strain windows"—periods where 'Net Daily Intake' was consistently positive (more children entering care than leaving) coinciding with 'High Load' conditions (above

the 90th percentile). These windows serve as critical indicators of potential system stress and resource challenges.

- **Efficiency Metrics:** The 'Discharge Offset Ratio', calculated as Children discharged from HHS Care / Children transferred out of CBP custody, provides a measure of system outflow efficiency. An average ratio of 1.34 suggests that, on average, more children are discharged from HHS care than are transferred in from CBP on any given day, though this can fluctuate.
- **System Stress and Pressure Identification:**
 - **Composite Stress Score:** A novel Composite Stress Score was developed, integrating 'Total System Load', its '7-Day Rolling Std Dev Load', and indicators for sustained net intake and high load. This score provides a robust, real-time measure of system strain.
 - **Composite Pressure Score:** Complementing stress, a Composite Pressure Score was also created. This score combines normalized Net Daily Intake, positive Care Load Growth Rate, and multipliers for sustained intake and high load, offering an early warning system for accumulating operational pressure.
- **Key Performance Indicators (KPIs):** A summary of key operational metrics provides a snapshot of system health:
 - **Total Children Under Care (Current/Average):** Current: 2,502; Average: 4,175. This highlights a recent lower load compared to the historical average.
 - **Current Net Intake Pressure:** -3, indicating a slight net outflow at the most recent data point.
 - **Care Load Volatility Index:** 41.88%, reflecting considerable variability in the system's load growth rate.
 - **Backlog Accumulation Rate (30-day avg):** 0.37, suggesting a marginal average net intake over the last month.
 - **Average Discharge Offset Ratio:** 1.34, further supporting a generally efficient discharge process relative to CBP transfers.

Recommendations for Future Action:

- **Investigate Data Anomalies:** Conduct a thorough investigation into instances where 'Children transferred out of CBP custody' exceeded 'Children in CBP custody' to clarify reporting mechanisms, data entry errors, or complex transfer dynamics that might not be fully captured.
- **Refine Missing Data Handling:** Explore more sophisticated imputation techniques (e.g., interpolation, machine learning-based imputation) for missing dates instead of solely relying on `fillna(0)`. This could provide a more realistic representation of periods with no reported data and prevent the introduction of artificial patterns that might skew model performance.
- **Further Develop Stress & Pressure Indices:** Continue to refine and rigorously validate the Composite Stress Score and Composite Pressure Score using

expert domain knowledge. Integrate these indices into real-time monitoring dashboards to provide proactive alerts for operational strain and inform adaptive resource allocation.

- **Integrate KPIs into Dashboards:** Develop an interactive dashboard to visualize all derived KPIs, allowing stakeholders to monitor system health, identify emerging trends, and track performance against thresholds over time. This dashboard should include the identified high-load periods and strain windows.
- **Root Cause Analysis for Strain Windows:** Conduct a deeper, qualitative and quantitative analysis into the specific factors contributing to the detected "prolonged strain windows." This could involve correlating these periods with external events, policy changes, or seasonal factors to identify underlying causes and develop targeted mitigation strategies.
- **Explore Advanced Time Series Decomposition:** Implement seasonal-trend decomposition techniques to more formally separate and analyze the seasonal, trend, and residual components of the 'Total System Load' and other key metrics. This can further refine understanding of temporal patterns before forecasting.

Recommendational Analysis Forecast

8. Feature Engineering for Time Series Forecasting

```
df_ml = df.copy()

# 1. Lag Features: To capture temporal dependencies
# Shift the 'Total System Load' to create lagged features
for i in range(1, 8): # Lag up to 7 days
    df_ml[f'Load_Lag_{i}d'] = df_ml['Total System Load'].shift(i)

# 2. Rolling Statistics: To capture trends and seasonality
# Rolling mean and standard deviation for 'Total System Load'
# Shift by 1 to prevent data leakage (rolling window should not
include current day's target)
for window in [7, 14, 30]: # Weekly, bi-weekly, monthly windows
    df_ml[f'Load_Rolling_Mean_{window}d'] = df_ml['Total System
Load'].rolling(window=window).mean().shift(1)
    df_ml[f'Load_Rolling_Std_{window}d'] = df_ml['Total System
Load'].rolling(window=window).std().shift(1)

# 3. Date-based Features: To capture cyclical patterns
df_ml['Day_of_Week'] = df_ml.index.dayofweek # Monday=0, Sunday=6
df_ml['Day_of_Month'] = df_ml.index.day
df_ml['Month'] = df_ml.index.month
df_ml['Year'] = df_ml.index.year
df_ml['Week_of_Year'] = df_ml.index.isocalendar().week.astype(int)
df_ml['Is_Weekend'] = (df_ml.index.dayofweek >= 5).astype(int) # 1 for
weekend, 0 for weekday
```

```

# Cyclical Features using Sine/Cosine Transformations
# Day of Week (cycle = 7)
df_ml['Day_of_Week_sin'] = np.sin(2 * np.pi * df_ml['Day_of_Week'] /
7)
df_ml['Day_of_Week_cos'] = np.cos(2 * np.pi * df_ml['Day_of_Week'] /
7)

# Month (cycle = 12)
df_ml['Month_sin'] = np.sin(2 * np.pi * df_ml['Month'] / 12)
df_ml['Month_cos'] = np.cos(2 * np.pi * df_ml['Month'] / 12)

# Day of Month (cycle = 31, approximate)
df_ml['Day_of_Month_sin'] = np.sin(2 * np.pi * df_ml['Day_of_Month'] /
31)
df_ml['Day_of_Month_cos'] = np.cos(2 * np.pi * df_ml['Day_of_Month'] /
31)

# 4. Interaction Features (Optional, based on domain knowledge)
# Example: Interaction between Net Daily Intake and Is_Weekend
df_ml['Net_Intake_x_Weekend'] = df_ml['Net Daily Intake'] *
df_ml['Is_Weekend']

# Replace any infinite values with NaN before dropping rows
df_ml.replace([np.inf, -np.inf], np.nan, inplace=True)

# Drop rows with NaN values resulting from feature creation (e.g., lag
features or replaced inf values)
df_ml.dropna(inplace=True)

# Display the first few rows of the DataFrame with new features
display(df_ml.head())

{"type": "dataframe"}

```

Lag Analysis

To ensure our predictive models are optimized, we visualize the relative importance of engineered features and the correlation decay of historical lags.

```

import plotly.express as px
import plotly.graph_objects as go

# Compute lag correlations
lags = [f'Load_Lag_{i}d' for i in range(1, 8)]
correlations = [df_ml['Total System Load'].corr(df_ml[lag]) for lag in
lags]

# Create interactive bar chart
fig = px.bar(

```

```

    x=list(range(1, 8)),
    y=correlations,
    labels={'x': 'Lag Period (Days)', 'y': 'Pearson Correlation'},
    title='Predictive Power Decay: Lag Correlation with Current Load',
    color=correlations, # color by correlation values
    color_continuous_scale='Viridis'
)

# Adjust y-axis limits dynamically
fig.update_yaxes(range=[min(correlations) - 0.05, 1.0])

# Layout tweaks
fig.update_layout(
    template='plotly_white',
    xaxis=dict(tickmode='linear'),
    title=dict(x=0.5, xanchor='center') # center the title
)

fig.show()

```

Rolling Statistics

```

import plotly.graph_objects as go

# 2. 3D Interaction: Rolling Mean, Volatility, and Actual Load
fig_interaction = go.Figure(data=[go.Scatter3d(
    x=df_ml['Load_Rolling_Mean_7d'],
    y=df_ml['Load_Rolling_Std_7d'],
    z=df_ml['Total System Load'],
    mode='markers',
    marker=dict(
        size=4,
        color=df_ml['Total System Load'],
        colorscale='Viridis',
        opacity=0.7,
        colorbar=dict(title='Actual Load')
    )
)])

fig_interaction.update_layout(
    title='Feature Interaction: Rolling Mean vs. Volatility vs. Target',
    scene=dict(
        xaxis_title='7d Rolling Mean',
        yaxis_title='7d Rolling Std Dev',
        zaxis_title='Current Load'
    )
)

fig_interaction.show()

```

Date-based Features

```
import plotly.express as px

# Map numeric days to labels
day_labels = {0: 'Mon', 1: 'Tue', 2: 'Wed', 3: 'Thu', 4: 'Fri', 5: 'Sat', 6: 'Sun'}
df_ml['Day_Label'] = df_ml['Day_of_Week'].map(day_labels).astype(str)
# force categorical
day_order = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

# Day of Week Distribution
fig_day = px.box(
    df_ml,
    x='Day_Label',
    y='Total System Load',
    points='all',
    color='Day_Label', # categorical coloring
    title='Load Distribution by Day of Week',
    labels={'Day_Label': 'Day of Week', 'Total System Load': 'Total System Load'},
    category_orders={'Day_Label': day_order},
    color_discrete_sequence=px.colors.qualitative.Set2 # categorical palette
)

fig_day.update_layout(
    template='plotly_white',
    hovermode='x unified',
    title=dict(x=0.5, xanchor='center')
)
fig_day.show()

# Ensure Month is numeric or ordered categorical
df_ml['Month'] = df_ml['Month'].astype(int) # if it's numeric (1-12)

# Create a mapping for month labels
month_labels = {
    1: 'Jan', 2: 'Feb', 3: 'Mar', 4: 'Apr',
    5: 'May', 6: 'Jun', 7: 'Jul', 8: 'Aug',
    9: 'Sep', 10: 'Oct', 11: 'Nov', 12: 'Dec'
}
df_ml['Month_Label'] = df_ml['Month'].map(month_labels)

# Explicit order for months
month_order = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

# Monthly Distribution with sequential order
fig_month = px.box(
```



```

df_ml,
x='Month_Label',
y='Total System Load',
points='all',
color='Month_Label',
title='Monthly Seasonality Patterns',
labels={'Month_Label': 'Month', 'Total System Load': 'Total System
Load'},
category_orders={'Month_Label': month_order}, # enforce
sequential order
color_discrete_sequence=px.colors.qualitative.Set3
)

fig_month.update_layout(
    template='plotly_white',
    hovermode='x unified',
    title=dict(x=0.5, xanchor='center')
)
fig_month.show()

```

8.1 Data Splitting for Time Series Forecasting

```

from sklearn.model_selection import train_test_split

# Define target variable and features
target_variable = 'Total System Load'
# Exclude 'Load_Category' from features, as it's a categorical target
for classification
features = [col for col in df_ml.columns if col not in
[target_variable, 'Load_Category', 'Day_Label', 'Month_Label']]

X = df_ml[features]
y = df_ml[target_variable]

# Increasing test size by reducing the training split to 70% (making
test 30%)
split_point = int(len(df_ml) * 0.7)

X_train = X.iloc[:split_point]
y_train = y.iloc[:split_point]
X_test = X.iloc[split_point:]
y_test = y.iloc[split_point:]

print(f"Training set size: {len(X_train)} samples")
print(f"Testing set size: {len(X_test)} samples")
print(f"Test set starts at: {X_test.index.min()}")

Training set size: 492 samples
Testing set size: 212 samples
Test set starts at: 2025-02-07 00:00:00

```

8.2 Create Classification Target Variable

```
import numpy as np

# Define percentiles for high and low load categories
# For 'High Load', we can use the 75th percentile as a threshold
high_load_threshold = df_ml['Total System Load'].quantile(0.75)

# For 'Low Load', we can use the 25th percentile as a threshold
low_load_threshold = df_ml['Total System Load'].quantile(0.25)

print(f"High Load Threshold (75th percentile):  
{high_load_threshold:.2f}")
print(f"Low Load Threshold (25th percentile):  
{low_load_threshold:.2f}")

# Create a new target variable for classification: 'Load_Category'
def get_load_category(load):
    if load >= high_load_threshold:
        return 'High'
    elif load <= low_load_threshold:
        return 'Low'
    else:
        return 'Medium'

df_ml['Load_Category'] = df_ml['Total System Load'].apply(get_load_category)

# Display the distribution of the new target variable
display(df_ml['Load_Category'].value_counts())
display(df_ml.head())

High Load Threshold (75th percentile): 8011.50
Low Load Threshold (25th percentile): 2062.75

Load_Category
Medium    352
High      176
Low       176
Name: count, dtype: int64

{"type": "dataframe"}
```

8.3 Data Splitting for Classification Task

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Define target variable and features for classification
target_variable_clf = 'Load_Category'
# Exclude 'Total System Load' as it's the regression target, and
```

```

'Day_Label', 'Month_Label' as they are string columns.
features_clf = [col for col in df_ml.columns if col not in
[target_variable_clf, 'Total System Load', 'Day_Label',
'Month_Label']]

X_clf = df_ml[features_clf]
y_clf = df_ml[target_variable_clf]

# Encode the categorical target variable
label_encoder = LabelEncoder()
y_clf_encoded = label_encoder.fit_transform(y_clf)

# Maintain consistency with the regression split (70/30)
split_point_clf = int(len(df_ml) * 0.7)

X_train_clf = X_clf.iloc[:split_point_clf]
y_train_clf = y_clf_encoded[:split_point_clf]
X_test_clf = X_clf.iloc[split_point_clf:]
y_test_clf = y_clf_encoded[split_point_clf:]

# Scale the features for classification models
scaler_clf = StandardScaler()
X_train_clf_scaled = scaler_clf.fit_transform(X_train_clf)
X_test_clf_scaled = scaler_clf.transform(X_test_clf)

# Convert scaled arrays back to DataFrames
X_train_clf_scaled = pd.DataFrame(X_train_clf_scaled,
columns=X_train_clf.columns, index=X_train_clf.index)
X_test_clf_scaled = pd.DataFrame(X_test_clf_scaled,
columns=X_test_clf.columns, index=X_test_clf.index)

print(f"Classification Training set size: {len(X_train_clf)} samples")
print(f"Classification Testing set size: {len(X_test_clf)} samples")

Classification Training set size: 492 samples
Classification Testing set size: 212 samples

```

9. Model Training and Evaluation

Before proceeding, please ensure you have executed the cells in **Section 7 (Feature Engineering)** and **Section 8 (Data Splitting)** to create `df_ml`, `X_train`, `y_train`, `X_test`, and `y_test`. If you encounter a `NameError`, it means those steps haven't been run or completed successfully.

9.1.1 Linear Regression

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import plotly.graph_objects as go

```

```

import plotly.express as px

# Initialize and train the Linear Regression model
lin_reg_model = LinearRegression()
lin_reg_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_lin_reg = lin_reg_model.predict(X_test)

# Evaluate the model
mae_lin_reg = mean_absolute_error(y_test, y_pred_lin_reg)
rmse_lin_reg = np.sqrt(mean_squared_error(y_test, y_pred_lin_reg))

print(f"Linear Regression MAE: {mae_lin_reg:.2f}")
print(f"Linear Regression RMSE: {rmse_lin_reg:.2f}")

# Plotly interactive chart
fig = go.Figure()

# Actual values
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_test,
    mode='lines',
    name='Actual Values',
    line=dict(color='blue')
))

# Predictions
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_pred_lin_reg,
    mode='lines',
    name='Linear Regression Predictions',
    line=dict(color='red', dash='dash')
))

# Layout
fig.update_layout(
    title=dict(
        text='Linear Regression: Actual vs. Predicted Total System
Load',
        x=0.5,
        xanchor='center'
    ),
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
)

```

```
)  
  
fig.show()  
  
Linear Regression MAE: 0.00  
Linear Regression RMSE: 0.00
```

9.1.2 XGBoost Regressor

```
import xgboost as xgb  
import plotly.graph_objects as go  
import numpy as np  
  
# Initialize and train the XGBoost Regressor model  
# Using some default parameters, can be tuned later  
xgb_reg_model = xgb.XGBRegressor(n_estimators=100, learning_rate=0.1,  
random_state=42)  
xgb_reg_model.fit(X_train, y_train)  
  
# Make predictions on the test set  
y_pred_xgb = xgb_reg_model.predict(X_test)  
  
# Evaluate the model  
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)  
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))  
  
print(f"XGBoost Regressor MAE: {mae_xgb:.2f}")  
print(f"XGBoost Regressor RMSE: {rmse_xgb:.2f}")  
  
# Plotly interactive chart  
fig = go.Figure()  
  
# Actual values  
fig.add_trace(go.Scatter(  
    x=y_test.index,  
    y=y_test,  
    mode='lines',  
    name='Actual Values',  
    line=dict(color='blue')  
))  
  
# Predictions  
fig.add_trace(go.Scatter(  
    x=y_test.index,  
    y=y_pred_xgb,  
    mode='lines',  
    name='XGBoost Predictions',  
    line=dict(color='green', dash='dash')  
))
```

```

# Layout
fig.update_layout(
    title=dict(
        text='XGBoost Regressor: Actual vs. Predicted Total System
Load',
        x=0.5,
        xanchor='center'
    ),
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
)

fig.show()

XGBoost Regressor MAE: 1718.57
XGBoost Regressor RMSE: 1972.33

```

9.1.3 Random Forest Regressor

```

from sklearn.ensemble import RandomForestRegressor
import plotly.graph_objects as go
import numpy as np

# Initialize and train the Random Forest Regressor model
# Using some default parameters, can be tuned later
rf_reg_model = RandomForestRegressor(n_estimators=100,
random_state=42)
rf_reg_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_reg_model.predict(X_test)

# Evaluate the model
mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

print(f"Random Forest Regressor MAE: {mae_rf:.2f}")
print(f"Random Forest Regressor RMSE: {rmse_rf:.2f}")

# Plotly interactive chart
fig = go.Figure()

# Actual values
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_test,
    mode='lines',

```

```

        name='Actual Values',
        line=dict(color='blue')
    ))

# Predictions
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_pred_rf,
    mode='lines',
    name='Random Forest Predictions',
    line=dict(color='purple', dash='dash')
))

# Layout
fig.update_layout(
    title=dict(
        text='Random Forest Regressor: Actual vs. Predicted Total
System Load',
        x=0.5,
        xanchor='center'
    ),
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
)

fig.show()

Random Forest Regressor MAE: 401.93
Random Forest Regressor RMSE: 581.34

```

9.1.4 Support Vector Regressor (SVR)

```

from sklearn.svm import SVR
import plotly.graph_objects as go
import numpy as np

# Initialize and train the SVR model
# Using some default parameters, can be tuned later. SVR can be
# computationally intensive.
svr_model = SVR(kernel='rbf', C=100, gamma=0.1) # RBF kernel, common
# starting point
svr_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_svr = svr_model.predict(X_test)

# Evaluate the model

```

```

mae_svr = mean_absolute_error(y_test, y_pred_svr)
rmse_svr = np.sqrt(mean_squared_error(y_test, y_pred_svr))

print(f"SVR Regressor MAE: {mae_svr:.2f}")
print(f"SVR Regressor RMSE: {rmse_svr:.2f}")

# Plotly interactive chart
fig = go.Figure()

# Actual values
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_test,
    mode='lines',
    name='Actual Values',
    line=dict(color='blue')
))

# Predictions
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_pred_svr,
    mode='lines',
    name='SVR Predictions',
    line=dict(color='orange', dash='dash')
))

# Layout
fig.update_layout(
    title=dict(
        text='SVR Regressor: Actual vs. Predicted Total System Load',
        x=0.5,
        xanchor='center'
    ),
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
)

fig.show()

SVR Regressor MAE: 5254.92
SVR Regressor RMSE: 5346.26

```

9.1.5 Gradient Boosting Regressor

```

from sklearn.ensemble import GradientBoostingRegressor
import plotly.graph_objects as go

```



```

import numpy as np

# Initialize and train the Gradient Boosting Regressor model
# Using some default parameters, can be tuned later
gbr_model = GradientBoostingRegressor(n_estimators=100,
learning_rate=0.1, max_depth=3, random_state=42)
gbr_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_gbr = gbr_model.predict(X_test)

# Evaluate the model
mae_gbr = mean_absolute_error(y_test, y_pred_gbr)
rmse_gbr = np.sqrt(mean_squared_error(y_test, y_pred_gbr))

print(f"Gradient Boosting Regressor MAE: {mae_gbr:.2f}")
print(f"Gradient Boosting Regressor RMSE: {rmse_gbr:.2f}")

# Plotly interactive chart
fig = go.Figure()

# Actual values
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_test,
    mode='lines',
    name='Actual Values',
    line=dict(color='blue')
))

# Predictions
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_pred_gbr,
    mode='lines',
    name='Gradient Boosting Predictions',
    line=dict(color='brown', dash='dash')
))

# Layout
fig.update_layout(
    title=dict(
        text='Gradient Boosting Regressor: Actual vs. Predicted Total
System Load',
        x=0.5,
        xanchor='center'
    ),
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',

```

```

        hovermode='x unified',
        legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
    )

fig.show()

Gradient Boosting Regressor MAE: 253.65
Gradient Boosting Regressor RMSE: 318.05

```

9.1.6 Decision Tree Regressor

Decision Tree Regressors are non-parametric supervised learning models that use a tree-like structure to make predictions. They split the data into branches based on feature values, with each leaf node representing a predicted value. These models are easy to interpret and can capture non-linear relationships.

```

from sklearn.tree import DecisionTreeRegressor
import plotly.graph_objects as go
import numpy as np

# Initialize and train the Decision Tree Regressor model
dt_reg_model = DecisionTreeRegressor(random_state=42)
dt_reg_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_dt = dt_reg_model.predict(X_test)

# Evaluate the model
mae_dt = mean_absolute_error(y_test, y_pred_dt)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

print(f"Decision Tree Regressor MAE: {mae_dt:.2f}")
print(f"Decision Tree Regressor RMSE: {rmse_dt:.2f}")

# Plotly interactive chart
fig = go.Figure()

# Actual values
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_test,
    mode='lines',
    name='Actual Values',
    line=dict(color='blue')
))

# Predictions
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_pred_dt,

```

```

        mode='lines',
        name='Decision Tree Predictions',
        line=dict(color='cyan', dash='dash')
    ))

# Layout
fig.update_layout(
    title=dict(
        text='Decision Tree Regressor: Actual vs. Predicted Total
System Load',
        x=0.5,
        xanchor='center'
    ),
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
)

fig.show()

Decision Tree Regressor MAE: 1662.82
Decision Tree Regressor RMSE: 1935.05

```

9.1.7 K-Neighbors Regressor

K-Neighbors Regressors are non-parametric methods that predict the value of a new data point based on the average of the values of its k nearest neighbors in the training data. This model is simple, but its performance highly depends on the choice of k and the distance metric used.

```

from sklearn.neighbors import KNeighborsRegressor
import plotly.graph_objects as go
import numpy as np

# Initialize and train the K-Neighbors Regressor model
knn_reg_model = KNeighborsRegressor(n_neighbors=5) # Using 5 neighbors
as a starting point
knn_reg_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_knn = knn_reg_model.predict(X_test)

# Evaluate the model
mae_knn = mean_absolute_error(y_test, y_pred_knn)
rmse_knn = np.sqrt(mean_squared_error(y_test, y_pred_knn))

print(f"K-Neighbors Regressor MAE: {mae_knn:.2f}")
print(f"K-Neighbors Regressor RMSE: {rmse_knn:.2f}")

```

```

# Plotly interactive chart
fig = go.Figure()

# Actual values
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_test,
    mode='lines',
    name='Actual Values',
    line=dict(color='blue')
))

# Predictions
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_pred_knn,
    mode='lines',
    name='K-Neighbors Predictions',
    line=dict(color='magenta', dash='dash')
))

# Layout
fig.update_layout(
    title=dict(
        text='K-Neighbors Regressor: Actual vs. Predicted Total System
Load',
        x=0.5,
        xanchor='center'
    ),
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
)

fig.show()

K-Neighbors Regressor MAE: 1310.95
K-Neighbors Regressor RMSE: 1429.66

```

9.1.8 Huber Regressor

Huber Regressors are robust regression models that are less sensitive to outliers in the data compared to ordinary least squares regression. It uses a loss function that is quadratic for small errors and linear for large errors, combining the best properties of L1 (LASSO) and L2 (Ridge) regularization. This makes it suitable for datasets with potentially noisy or outlying observations.

```

from sklearn.linear_model import HuberRegressor
from sklearn.preprocessing import StandardScaler
import plotly.graph_objects as go
import numpy as np

# Initialize and train the Huber Regressor model
huber_reg_model = HuberRegressor(max_iter=5000, alpha=0.0001) #
Increased max_iter

# Scale the features for the regression task
scaler_reg = StandardScaler()
X_train_scaled = scaler_reg.fit_transform(X_train)
X_test_scaled = scaler_reg.transform(X_test)

huber_reg_model.fit(X_train_scaled, y_train)

# Make predictions on the test set (using scaled features)
y_pred_huber = huber_reg_model.predict(X_test_scaled)

# Evaluate the model
mae_huber = mean_absolute_error(y_test, y_pred_huber)
rmse_huber = np.sqrt(mean_squared_error(y_test, y_pred_huber))

print(f"Huber Regressor MAE: {mae_huber:.2f}")
print(f"Huber Regressor RMSE: {rmse_huber:.2f}")

# Plotly interactive chart
fig = go.Figure()

# Actual values
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_test,
    mode='lines',
    name='Actual Values',
    line=dict(color='blue')
))

# Predictions
fig.add_trace(go.Scatter(
    x=y_test.index,
    y=y_pred_huber,
    mode='lines',
    name='Huber Regressor Predictions',
    line=dict(color='lime', dash='dash')
))

# Layout
fig.update_layout(
    title=dict(
        text='Huber Regressor: Actual vs. Predicted Total System

```

```

Load',
    x=0.5,
    xanchor='center'
),
xaxis_title='Date',
yaxis_title='Total System Load',
template='plotly_white',
hovermode='x unified',
legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1)
)

fig.show()

Huber Regressor MAE: 0.00
Huber Regressor RMSE: 0.00

```

9.1.9 Model Performance Comparison Heatmap

```

import pandas as pd
import plotly.graph_objects as go
import plotly.express as px

# Collect all MAE and RMSE values
performance_data = {
    'MAE': {
        'Linear Regression': mae_lin_reg,
        'XGBoost Regressor': mae_xgb,
        'Random Forest Regressor': mae_rf,
        'SVR': mae_svr,
        'Gradient Boosting Regressor': mae_gbr,
        'Decision Tree Regressor': mae_dt, # Added new model
        'K-Neighbors Regressor': mae_knn, # Added new model
        'Huber Regressor': mae_huber # Added new model
    },
    'RMSE': {
        'Linear Regression': rmse_lin_reg,
        'XGBoost Regressor': rmse_xgb,
        'Random Forest Regressor': rmse_rf,
        'SVR': rmse_svr,
        'Gradient Boosting Regressor': rmse_gbr,
        'Decision Tree Regressor': rmse_dt, # Added new model
        'K-Neighbors Regressor': rmse_knn, # Added new model
        'Huber Regressor': rmse_huber # Added new model
    }
}

performance_df = pd.DataFrame(performance_data)

print("\n--- Model Performance Summary ---")
display(performance_df.sort_values(by='MAE'))

```

```

# Plotting the heatmap using Plotly
fig_heatmap = go.Figure(data=go.Heatmap(
    z=performance_df.values,
    x=performance_df.columns,
    y=performance_df.index,
    colorscale='Viridis_r', # Use _r to reverse and show lower
values as 'hotter'/'better'
    text=performance_df.values,
    texttemplate="%{text:.2f}",
    hovertemplate="Model: %{y}<br>Metric: %{x}<br>Value: %
{z:.2f}<extra></extra>"
))

fig_heatmap.update_layout(
    title=dict(
        text='<b>Regression Model Performance Comparison (MAE &
RMSE)</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    xaxis_title='Metric',
    yaxis_title='Model',
    template='plotly_white',
    height=700
)

fig_heatmap.show()

# Update predictions_df to include new models
predictions_df = pd.DataFrame({
    'Actual': y_test,
    'Linear Regression': y_pred_lin_reg,
    'XGBoost': y_pred_xgb,
    'Random Forest': y_pred_rf,
    'SVR': y_pred_svr,
    'Gradient Boosting': y_pred_gbr,
    'Decision Tree': y_pred_dt, # Added new model
    'K-Neighbors': y_pred_knn, # Added new model
    'Huber Regressor': y_pred_huber # Added new model
}, index=y_test.index)
predictions_df.index.name = 'Date'
predictions_df.to_csv('model_predictions.csv')

```

--- Model Performance Summary ---

```

{"summary":{"\n  \"name\": \"predictions_df\", \n  \"rows\": 8, \n
\"fields\": [\n    {\n      \"column\": \"MAE\", \n

```

```

{"properties": {"dtype": "number", "std": 1741.2871748475409, "min": 2.0755353196392492e-12, "max": 5254.917714884696, "num_unique_values": 8, "samples": [9.060430621279663e-07, 1662.816037735849, 2.0755353196392492e-12]}, "semantic_type": "", "description": "", "column": "RMSE", "properties": {"dtype": "number", "std": 1769.994420905413, "min": 2.463727585308298e-12, "max": 5346.255608824026, "num_unique_values": 8, "samples": [1.0026544146089214e-06, 1935.0460637586705, 2.463727585308298e-12]}, "semantic_type": "", "description": ""}, {"type": "dataframe"}

```

9.1.10 3D Scatter Plot: Regression Model Performance Comparison

This 3D scatter plot visualizes the performance of different regression models. Each point represents a model, with its position determined by Mean Absolute Error (MAE) on the X-axis, Root Mean Squared Error (RMSE) on the Y-axis, and the model name on the Z-axis (though represented visually as distinct points). The points are colored based on their RMSE values, allowing for quick identification of models with lower (better) RMSE.

```

import plotly.graph_objects as go
import pandas as pd

# Ensure performance_df is available from previous steps
# (Assuming performance_df has 'MAE' and 'RMSE' and Model names as index)

fig_regression_3d = go.Figure(
    data=[
        go.Scatter3d(
            x=performance_df['MAE'],
            y=performance_df['RMSE'],
            z=[f'{model}' for model in performance_df.index], # Using
            model names for Z-axis labels
            mode='markers',
            marker=dict(
                size=8,
                color=performance_df['RMSE'], # Color by RMSE
                colorscale='Viridis_r', # Viridis_r to show lower RMSE
                as 'hotter' or more prominent
                opacity=0.8,
                colorbar=dict(
                    title='<b>RMSE</b>',
                    x=1.05
                )
            ),
            text=[f'Model: {model}<br>MAE: {mae:.2f}<br>RMSE:

```



```

{rmse:.2f}'
                for model, mae, rmse in zip(performance_df.index,
performance_df['MAE'], performance_df['RMSE'])),
                hoverinfo='text'
            )
        ]
    )

fig_regression_3d.update_layout(
    title=dict(
        text='<b>3D Scatter Plot: Regression Model Performance (MAE vs
RMSE)</b>', # Bold, centered, size 17
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='MAE',
        yaxis_title='RMSE',
        zaxis_title='Model',
        zaxis=dict(tickmode='array',
tickvals=list(range(len(performance_df.index))),
ticktext=performance_df.index.tolist()) # Set explicit tick labels for
models
    ),
    height=700
)

fig_regression_3d.show()

```

9.1.11 3D Heatmap Surface Plot: Regression Model Performance Across Metrics

This plot visualizes the performance of each regression model across Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) in a 3D representation. The X-axis is dedicated to models, the Y-axis to metrics, and the Z-axis (and color) shows the performance score. This provides a comprehensive overview of how each model performs on different aspects of regression.

```

import plotly.graph_objects as go
import pandas as pd
import numpy as np
from scipy.interpolate import griddata

# Ensure performance_df is available from previous steps and has
'Model' as index
# Reset index to make 'Model' a column for melting
performance_df_reset =
performance_df.reset_index().rename(columns={'index': 'Model'})

```

```

# Melt the DataFrame to have 'Model', 'Metric', 'Value'
df_melted_regression_performance = performance_df_reset.melt(
    id_vars=['Model'],
    var_name='Metric',
    value_name='Score'
)

# Create numerical mappings for Model and Metric for plotting
model_names_reg = df_melted_regression_performance['Model'].unique()
metric_names_reg = df_melted_regression_performance['Metric'].unique()

model_mapping_reg = {model: i for i, model in
    enumerate(model_names_reg)}
metric_mapping_reg = {metric: i for i, metric in
    enumerate(metric_names_reg)}

df_melted_regression_performance['Model_Numeric'] =
df_melted_regression_performance['Model'].map(model_mapping_reg)
df_melted_regression_performance['Metric_Numeric'] =
df_melted_regression_performance['Metric'].map(metric_mapping_reg)

# Prepare data for surface plot
x_coords_reg =
df_melted_regression_performance['Model_Numeric'].values
y_coords_reg =
df_melted_regression_performance['Metric_Numeric'].values
z_values_reg = df_melted_regression_performance['Score'].values

# Create a denser grid for interpolation to allow for a smoother
surface and a clear gap
# Span x-axis from min_model_numeric to max_model_numeric
# Span y-axis from min_metric_numeric to max_metric_numeric
grid_x_reg, grid_y_reg = np.mgrid[
    x_coords_reg.min():x_coords_reg.max():50j, # More points for
models
    y_coords_reg.min():y_coords_reg.max():50j # More points for
metrics
]

# Interpolate the z_values (Score) onto the grid
grid_z_reg = griddata(
    (x_coords_reg, y_coords_reg),
    z_values_reg,
    (grid_x_reg, grid_y_reg),
    method='cubic' # Use cubic for smoother surface, linear can also
work
)

# --- Create the transparent distinguishable wall ---

```

```

# Identify the y-coordinates in the grid that are in the "middle"
between MAE (0) and RMSE (1)
# Define a narrow band around y=0.5
wall_y_start = 0.45 # Start of the transparent wall band
wall_y_end = 0.55 # End of the transparent wall band

# Find indices in grid_y_reg where y-coordinates fall within the wall
band
# np.where returns a tuple of arrays, one for each dimension. We need
the 2D indices.
wall_indices_y_dim = np.where((grid_y_reg >= wall_y_start) &
(grid_y_reg <= wall_y_end))

# Set the z-values within this band to NaN to create a transparent
gap/wall
grid_z_reg[wall_indices_y_dim] = np.nan

# Create the 3D surface plot
fig_regression_surface_3d = go.Figure(data=[
    go.Surface(
        z=grid_z_reg,
        x=grid_x_reg,
        y=grid_y_reg,
        colorscale='Viridis_r', # Use 'Viridis_r' to highlight lower
scores (better) with warmer colors
        colorbar=dict(
            title='<b>Performance Score</b>',
            x=1.05
        ),
        cmin=z_values_reg.min(),
        cmap=z_values_reg.max()
    )
])

fig_regression_surface_3d.update_layout(
    title=dict(
        text='<b>3D Heatmap Surface Plot: Regression Model Performance
(with wall)</b>', # Updated title
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Model',
        yaxis_title='Metric',
        zaxis_title='Performance Score',
        xaxis=dict(tickmode='array',
tickvals=list(model_mapping_reg.values()),
ticktext=list(model_mapping_reg.keys())),
        yaxis=dict(tickmode='array',

```

```

tickvals=list(metric_mapping_reg.values()),
ticktext=list(metric_mapping_reg.keys()))
    ),
    height=700
)

fig_regression_surface_3d.show()

```

9.2 Classification Model Training and Evaluation

```

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score
from sklearn.multiclass import OneVsRestClassifier
import xgboost as xgb
import numpy as np
import pandas as pd
import warnings # Import the warnings module

# Initialize classifiers
classifiers = {
    'Logistic Regression':
OneVsRestClassifier(LogisticRegression(random_state=42,
solver='liblinear')),
    'Random Forest': RandomForestClassifier(random_state=42),
    'XGBoost': xgb.XGBClassifier(eval_metric='mlogloss',
random_state=42),
    'SVC': SVC(probability=True, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'K-Neighbors': KNeighborsClassifier(),
    'Gaussian Naive Bayes': GaussianNB()
}
results = []

# Get all possible encoded labels from the label_encoder
# This ensures roc_auc_score is aware of all classes, even if some are
missing in y_test_clf
all_encoded_labels = label_encoder.transform(label_encoder.classes_)

for name, clf in classifiers.items():
    print(f"\nTraining {name}...")
    # Train with scaled data
    clf.fit(X_train_clf_scaled, y_train_clf)

```

```

y_pred = clf.predict(X_test_clf_scaled)

accuracy = accuracy_score(y_test_clf, y_pred)
# For multi-class, use 'weighted' or 'macro' for precision,
recall, f1-score
precision = precision_score(y_test_clf, y_pred,
average='weighted', zero_division=0)
recall = recall_score(y_test_clf, y_pred, average='weighted',
zero_division=0)
f1 = f1_score(y_test_clf, y_pred, average='weighted',
zero_division=0)

# ROC AUC for multi-class needs prediction probabilities
# Check if y_test_clf has more than one unique class for ROC AUC
calculation
unique_classes_in_y_test = np.unique(y_test_clf)
if len(unique_classes_in_y_test) > 1 and hasattr(clf,
"predict_proba"):
    y_proba = clf.predict_proba(X_test_clf_scaled)
    # Suppress UndefinedMetricWarning for roc_auc_score when
y_true has only one class
    with warnings.catch_warnings():
        warnings.filterwarnings("ignore", message="Only one class
is present in y_true.", category=UserWarning)
        roc_auc = roc_auc_score(y_test_clf, y_proba,
multi_class='ovr', average='weighted', labels=all_encoded_labels)
    else:
        roc_auc = np.nan # Not all classifiers support predict_proba,
or only one class present

    results.append({
        'Model': name,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1,
        'ROC AUC': roc_auc
    })

classification_performance_df = pd.DataFrame(results)
display(classification_performance_df.sort_values(by='F1-Score',
ascending=False))

```

Training Logistic Regression...

Training Random Forest...

Training XGBoost...

Training SVC...

Training Gradient Boosting...

Training Decision Tree...

Training K-Neighbors...

Training Gaussian Naive Bayes...

```
{"summary":{"\n  \"name\": \"display(classification_performance_df\", \n  \"rows\": 8, \n  \"fields\": [\n    {\n      \"column\": \"Model\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 8, \n        \"samples\": [\n          \"Decision Tree\", \n          \"K-Neighbors\", \n          \"Gaussian Naive Bayes\", \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          } \n        }, \n        {\n      \"column\": \"Accuracy\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.1967406890905732, \n        \"min\": 0.32547169811320753, \n        \"max\": 0.910377358490566, \n        \"num_unique_values\": 7, \n        \"samples\": [\n          0.910377358490566, \n          0.9009433962264151, \n          0.6792452830188679 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          } \n        }, \n        {\n      \"column\": \"Precision\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.27797347705505127, \n        \"min\": 0.1059318262726949, \n        \"max\": 0.937543675751223, \n        \"num_unique_values\": 7, \n        \"samples\": [\n          0.9208886559515491, \n          0.9073816219158201, \n          0.7826164714298489 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          } \n        }, \n        {\n      \"column\": \"Recall\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.1967406890905732, \n        \"min\": 0.32547169811320753, \n        \"max\": 0.910377358490566, \n        \"num_unique_values\": 7, \n        \"samples\": [\n          0.910377358490566, \n          0.9009433962264151, \n          0.6792452830188679 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          } \n        }, \n        {\n      \"column\": \"F1-Score\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.255960487327394, \n        \"min\": 0.15984019337944, \n        \"max\": 0.9060141346981107, \n        \"num_unique_values\": 7, \n        \"samples\": [\n          0.9060141346981107, \n          0.896763690091136, \n          0.5542570849894163 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          } \n        }, \n        {\n      \"column\": \"ROC AUC\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.03892450503657604, \n        \"min\": 0.8095285217926727, \n        \"max\": 0.9229707945868734, \n        \"num_unique_values\": 7, \n        \"samples\": [\n          0.8623188405797102, \n          0.8636645689557914, \n          0.8372352285395763 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          } \n        } \n      ] \n    } \n  } \n}
```

```
\ "semantic_type\": \ "\",\n          \ "description\": \ "\ "\n      }\n      ]\n    }", "type": "dataframe"}
```

9.2.1 Classification Model Performance Comparison Heatmap

```
import plotly.graph_objects as go
import pandas as pd

# Ensure classification_performance_df is available from previous
# steps
# (Assuming classification_performance_df has 'Model' as a column and
# metrics as other columns)

# Set 'Model' as index for the heatmap to ensure it's on the y-axis
performance_df_clf_indexed =
classification_performance_df.set_index('Model')

fig_heatmap_clf = go.Figure(data=go.Heatmap(
    z=performance_df_clf_indexed.values,
    x=performance_df_clf_indexed.columns,
    y=performance_df_clf_indexed.index,
    colorscale='Viridis', # Using Viridis, as higher scores are
    # generally better for classification metrics
    text=performance_df_clf_indexed.values,
    texttemplate="%{text:.3f}",
    hovertemplate="Model: %{y}<br>Metric: %{x}<br>Value: %
    {z:.3f}<extra></extra>"
))

fig_heatmap_clf.update_layout(
    title=dict(
        text='<b>Classification Model Performance Comparison</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    xaxis_title='Metric',
    yaxis_title='Model',
    template='plotly_white',
    height=700
)

fig_heatmap_clf.show()
```

9.2.2 3D Heatmap Surface Plot: Classification Model Performance Across Metrics

This plot visualizes the performance of each classification model across various metrics (Accuracy, Precision, Recall, F1-Score, ROC AUC) in a 3D representation. Each point represents a

specific model-metric combination, with the X-axis dedicated to models, the Y-axis to metrics, and the Z-axis (and color) showing the performance score. This provides a comprehensive overview of how each model performs on different aspects of classification.

```
import plotly.graph_objects as go
import pandas as pd
import numpy as np

# Ensure classification_performance_df is available
# Melt the DataFrame to have 'Model', 'Metric', 'Value'
df_melted_performance = classification_performance_df.melt(
    id_vars=['Model'],
    var_name='Metric',
    value_name='Score'
)

# Create numerical mappings for Model and Metric for plotting
model_names = df_melted_performance['Model'].unique()
metric_names = df_melted_performance['Metric'].unique()

model_mapping = {model: i for i, model in enumerate(model_names)}
metric_mapping = {metric: i for i, metric in enumerate(metric_names)}

df_melted_performance['Model_Numeric'] =
df_melted_performance['Model'].map(model_mapping)
df_melted_performance['Metric_Numeric'] =
df_melted_performance['Metric'].map(metric_mapping)

# Filter out NaN scores for plotting, if any (e.g., ROC AUC for single
class)
df_plot_3d_heatmap = df_melted_performance.dropna(subset=['Score'])

fig_3d_heatmap = go.Figure(data=[
    go.Scatter3d(
        x=df_plot_3d_heatmap['Model_Numeric'],
        y=df_plot_3d_heatmap['Metric_Numeric'],
        z=df_plot_3d_heatmap['Score'],
        mode='markers',
        marker=dict(
            size=10,
            color=df_plot_3d_heatmap['Score'], # Color by score
            colorscale='Viridis', # Choose a colorscale where
higher is better (e.g., green/yellow)
            opacity=0.8,
            colorbar=dict(
                title='<b>Performance Score</b>',
                x=1.05
            )
        ),
    ),
    text=[
```



```

        f'Model: {model}<br>Metric: {metric}<br>Score:
{score:.3f}']
        for model, metric, score in zip(
            df_plot_3d_heatmap['Model'],
            df_plot_3d_heatmap['Metric'],
            df_plot_3d_heatmap['Score']
        )
    ],
    hoverinfo='text'
)
])

fig_3d_heatmap.update_layout(
    title=dict(
        text='<b>3D Scatter Plot: Classification Model
Performance</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Model',
        yaxis_title='Metric',
        zaxis_title='Performance Score',
        xaxis=dict(tickmode='array',
tickvals=list(model_mapping.values()),
ticktext=list(model_mapping.keys()),
        yaxis=dict(tickmode='array',
tickvals=list(metric_mapping.values()),
ticktext=list(metric_mapping.keys()))
    ),
    height=700
)

fig_3d_heatmap.show()

```

9.2.3 3D Heatmap Surface Plot: Classification Model Performance Across Metrics

This plot visualizes the performance of each classification model across various metrics (Accuracy, Precision, Recall, F1-Score, ROC AUC) in a 3D representation. Each point represents a specific model-metric combination, with the X-axis dedicated to models, the Y-axis to metrics, and the Z-axis (and color) showing the performance score. This provides a comprehensive overview of how each model performs on different aspects of classification.

```

import plotly.graph_objects as go
import pandas as pd
import numpy as np
from scipy.interpolate import griddata

```

```

# Ensure classification_performance_df is available
# Melt the DataFrame to have 'Model', 'Metric', 'Value'
df_melted_performance = classification_performance_df.melt(
    id_vars=['Model'],
    var_name='Metric',
    value_name='Score'
)

# Create numerical mappings for Model and Metric for plotting
model_names = df_melted_performance['Model'].unique()
metric_names = df_melted_performance['Metric'].unique()

model_mapping = {model: i for i, model in enumerate(model_names)}
metric_mapping = {metric: i for i, metric in enumerate(metric_names)}

df_melted_performance['Model_Numeric'] =
df_melted_performance['Model'].map(model_mapping)
df_melted_performance['Metric_Numeric'] =
df_melted_performance['Metric'].map(metric_mapping)

# Filter out NaN scores for plotting, if any (e.g., ROC AUC for single
class)
df_plot_3d_heatmap = df_melted_performance.dropna(subset=['Score'])

# --- Prepare data for surface plot ---
x_coords_clf = df_plot_3d_heatmap['Model_Numeric'].values
y_coords_clf = df_plot_3d_heatmap['Metric_Numeric'].values
z_values_clf = df_plot_3d_heatmap['Score'].values

# Create a grid for interpolation
grid_x_clf, grid_y_clf = np.mgrid[
    x_coords_clf.min():x_coords_clf.max():50j,
    y_coords_clf.min():y_coords_clf.max():50j
]

# Interpolate the z_values (Score) onto the grid
grid_z_clf = griddata(
    (x_coords_clf, y_coords_clf),
    z_values_clf,
    (grid_x_clf, grid_y_clf),
    method='cubic'
)

# Create the 3D surface plot
fig_3d_heatmap_surface = go.Figure(data=[
    go.Surface(
        z=grid_z_clf,
        x=grid_x_clf,
        y=grid_y_clf,

```

```

        colorscale='Viridis', # Choose a colorscale where higher is
better (e.g., green/yellow)
        colorbar=dict(
            title='<b>Performance Score</b>',
            x=1.05
        ),
        cmin=z_values_clf.min(),
        cmap=z_values_clf.max()
    )
])

fig_3d_heatmap_surface.update_layout(
    title=dict(
        text='<b>3D Heatmap Surface Plot: Classification Model
Performance</b>', # Updated title
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    scene=dict(
        xaxis_title='Model',
        yaxis_title='Metric',
        zaxis_title='Performance Score',
        xaxis=dict(tickmode='array',
tickvals=list(model_mapping.values()),
ticktext=list(model_mapping.keys())),
        yaxis=dict(tickmode='array',
tickvals=list(metric_mapping.values()),
ticktext=list(metric_mapping.keys()))
    ),
    height=700
)

fig_3d_heatmap_surface.show()

```

9.3 Best Model Identification

Based on the performance metrics, we can identify the best models for both the regression and classification tasks.

```

import pandas as pd

# Best Regression Model
best_regression_model = performance_df.sort_values(by='MAE').iloc[0]
print("--- Best Regression Model (Lowest MAE) ---")
display(best_regression_model)

# Best Classification Model
# Sort by F1-Score (highest first), then by Accuracy (highest first),

```

```

and then by Recall (highest first)
best_classification_model_re_evaluated =
classification_performance_df.sort_values(
    by=['F1-Score', 'Accuracy', 'Recall'], ascending=[False, False,
False]
).iloc[0]

print("--- Best Classification Model (Sorted by F1-Score, Accuracy,
then Recall) ---")
display(best_classification_model_re_evaluated)

--- Best Regression Model (Lowest MAE) ---

MAE      2.075535e-12
RMSE     2.463728e-12
Name: Linear Regression, dtype: float64

--- Best Classification Model (Sorted by F1-Score, Accuracy, then
Recall) ---

Model      Decision Tree
Accuracy      0.910377
Precision     0.920889
Recall        0.910377
F1-Score      0.906014
ROC AUC       0.862319
Name: 5, dtype: object

```

10. Enhance System Stress Monitoring

This section develops advanced monitoring metrics to provide early warnings for operational strain. We will implement an **Operational Pressure Index (OPI)** that looks at the velocity of intake relative to the standard deviation of discharge.

```

import numpy as np

# Calculate Inflow Velocity (3-day vs 10-day comparison)
df['Inflow_Velocity'] = df['Children transferred out of CBP
custody'].rolling(window=3).mean() - df['Children transferred out of
CBP custody'].rolling(window=10).mean()

# Operational Pressure Index: Net Intake normalized by 7-day
volatility
df['Operational_Pressure_Index'] = (df['Net Daily
Intake'].rolling(window=7).mean() / (df['7-Day Rolling Std Dev Load']
+ 1))

# Flag high pressure events (OPI > 1 standard deviation above mean)
opi_mean = df['Operational_Pressure_Index'].mean()
opi_std = df['Operational_Pressure_Index'].std()

```

[illegible]

```
print("Advanced Stress Metrics Calculated.")
display(df[['Inflow_Velocity', 'Operational_Pressure_Index',
            'High_Pressure_Alert']].tail(10))
```

Advanced Stress Metrics Calculated.

```
{
  "summary": {
    "name": "display(df[['Inflow_Velocity',
'Operational_Pressure_Index', 'High_Pressure_Alert']]",
    "rows": 10,
    "fields": [
      {
        "column": "Date",
        "properties": {
          "dtype": "date",
          "min": "2025-12-12 00:00:00",
          "max": "2025-12-21 00:00:00",
          "num_unique_values": 10,
          "samples": [
            "2025-12-20 00:00:00",
            "2025-12-13 00:00:00",
            "2025-12-17 00:00:00"
          ],
          "semantic_type": "\"",
          "description": "\"\""}
        },
        "column": "Inflow_Velocity",
        "properties": {
          "dtype": "number",
          "std": 2.930840691763001,
          "min": -3.4000000000000004,
          "max": 4.766666666666666,
          "num_unique_values": 10,
          "samples": [
            "3.4000000000000004",
            "-2.0",
            "4.766666666666666"
          ],
          "semantic_type": "\"",
          "description": "\"\""}
        },
        "column": "Operational_Pressure_Index",
        "properties": {
          "dtype": "number",
          "std": 0.00041428472821490413,
          "min": -0.0009391296672880841,
          "max": 0.0004670171810331006,
          "num_unique_values": 7,
          "samples": [
            "0.0",
            "-0.000589088379075089",
            "-0.0005820227903470768"
          ],
          "semantic_type": "\"",
          "description": "\"\""}
        },
        "column": "High_Pressure_Alert",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 0,
          "num_unique_values": 1,
          "samples": [
            "0"
          ],
          "semantic_type": "\"",
          "description": "\"\""}
        }
      ]
    },
    "type": "dataframe"
  }
}
```

```
import plotly.graph_objects as go
import pandas as pd
import numpy as np
```

```
fig = go.Figure()
```

```
# Add the OPI line
fig.add_trace(go.Scatter(
    x=df.index,
    y=df['Operational_Pressure_Index'],
    mode='lines',
    name='Operational Pressure Index',
    line=dict(color='crimson')
))
```

```

))

# Add the alert threshold line
fig.add_hline(
    y=opi_mean + opi_std,
    line_dash="dash",
    line_color="red",
    annotation_text=f"Alert Threshold ({opi_mean + opi_std:.2f})",
    annotation_position="top left",
    name='Alert Threshold'
)

# Create a masked series for the fill area (Strain Events)
y_fill =
df['Operational_Pressure_Index'].where(df['High_Pressure_Alert'] == 1,
np.nan)

# Add the strain events as a filled area
fig.add_trace(go.Scatter(
    x=df.index,
    y=y_fill,
    mode='lines',
    fill='tozeroy',
    fillcolor='rgba(255, 0, 0, 0.3)',
    line=dict(color='rgba(255, 0, 0, 0)'), # Make line invisible
    name='Strain Event (Alert Period)'
))

# Update layout
fig.update_layout(
    title=dict(
        text='<b>Operational Pressure Index (OPI) Over Time</b>',
        x=0.5,
        xanchor='center',
        font=dict(size=17)
    ),
    xaxis_title='Date',
    yaxis_title='Pressure Score',
    template='plotly_white',
    hovermode='x unified',
    height=600
)

fig.show()

```

10.1 Future Predictions using OpenAI-compatible Model

This section will generate future predictions using the best performing regression model (Gradient Boosting Regressor) from our evaluation, simulating how an OpenAI-compatible model might be used for forecasting within a broader system.

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# -----
# Parameters
# -----
months_to_predict = 18 # set horizon manually (0-24 Months Suitable)
future_steps = int(months_to_predict * 30.4) # approx days

print(f" Forecasting for the next {months_to_predict} months
(~{future_steps} days).")

# -----
# Add cyclical features to historical dataset
# -----
df_ml['Day_of_Week_sin'] = np.sin(2 * np.pi * df_ml['Day_of_Week'] /
7)
df_ml['Day_of_Week_cos'] = np.cos(2 * np.pi * df_ml['Day_of_Week'] /
7)
df_ml['Month_sin'] = np.sin(2 * np.pi * df_ml['Month'] / 12)
df_ml['Month_cos'] = np.cos(2 * np.pi * df_ml['Month'] / 12)
df_ml['Day_of_Month_sin'] = np.sin(2 * np.pi * df_ml['Day_of_Month'] /
31)
df_ml['Day_of_Month_cos'] = np.cos(2 * np.pi * df_ml['Day_of_Month'] /
31)

# -----
# Prepare dataset for re-training (date/exogenous features only)
# -----
X_full =
df_ml[['Day_of_Week', 'Day_of_Month', 'Month', 'Year', 'Week_of_Year', 'Is_
Weekend',

'Day_of_Week_sin', 'Day_of_Week_cos', 'Month_sin', 'Month_cos',
'Day_of_Month_sin', 'Day_of_Month_cos']]

y_full = df_ml['Total System Load']

# Handle NaNs
imputer = SimpleImputer(strategy="mean")
```

```

X_full_imputed = imputer.fit_transform(X_full)

# Scale features
scaler_forecast = StandardScaler()
X_full_scaled = scaler_forecast.fit_transform(X_full_imputed)

# Train GBR model
gbr_model_retrained = GradientBoostingRegressor(
    n_estimators=1000, learning_rate=1, max_depth=3, random_state=42
)
gbr_model_retrained.fit(X_full_scaled, y_full)

# -----
# Build future feature matrix
# -----
last_date = df_ml.index[-1]
future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1),
                             periods=future_steps, freq='D')

future_X = pd.DataFrame(index=future_dates)
future_X['Day_of_Week'] = future_X.index.dayofweek
future_X['Day_of_Month'] = future_X.index.day
future_X['Month'] = future_X.index.month
future_X['Year'] = future_X.index.year
future_X['Week_of_Year'] =
future_X.index.isocalendar().week.astype(int)
future_X['Is_Weekend'] = (future_X.index.dayofweek >= 5).astype(int)

future_X['Day_of_Week_sin'] = np.sin(2 * np.pi *
future_X['Day_of_Week'] / 7)
future_X['Day_of_Week_cos'] = np.cos(2 * np.pi *
future_X['Day_of_Week'] / 7)
future_X['Month_sin'] = np.sin(2 * np.pi * future_X['Month'] / 12)
future_X['Month_cos'] = np.cos(2 * np.pi * future_X['Month'] / 12)
future_X['Day_of_Month_sin'] = np.sin(2 * np.pi *
future_X['Day_of_Month'] / 31)
future_X['Day_of_Month_cos'] = np.cos(2 * np.pi *
future_X['Day_of_Month'] / 31)

# -----
# Direct forecast (no iterative loop)
# -----
future_X_imputed = imputer.transform(future_X)
future_X_scaled = scaler_forecast.transform(future_X_imputed)
future_forecast = gbr_model_retrained.predict(future_X_scaled)

future_forecast = pd.Series(future_forecast, index=future_dates)
future_forecast = np.clip(future_forecast, 0, None) # enforce non-
negative

```



```

print(f" Generated {len(future_forecast)} direct future
predictions.")

# -----
# Plot historical + forecast
# -----
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=df_ml.index,
    y=df_ml['Total System Load'],
    mode='lines',
    name='Historical Actuals',
    line=dict(color='blue')
))

fig.add_trace(go.Scatter(
    x=future_forecast.index,
    y=future_forecast,
    mode='lines',
    name='Direct Forecast (GBR Model)',
    line=dict(color='red', dash='dash')
))

fig.update_layout(
    title='Total System Load: Historical Data and Direct Forecast',
    xaxis_title='Date',
    yaxis_title='Total System Load',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(x=0.01, y=0.99, bordercolor="Black", borderwidth=1),
    height=600
)

fig.show()

 Forecasting for the next 18 months (~547 days).
 Generated 547 direct future predictions.

```

11. Conclusion

This notebook embarked on a comprehensive analysis of the HHS Unaccompanied Alien Children Program data, covering data ingestion, quality validation, feature engineering, and the application of various machine learning models for forecasting and period identification.

Key Findings:

- **Data Quality:** Initial data cleaning addressed missing dates and non-numeric entries. However, anomalies such as 'Children transferred out of CBP custody'

exceeding 'Children in CBP custody' were identified, suggesting potential reporting issues.

- **Derived Metrics:** Several insightful metrics were created, including 'Total System Load', 'Net Daily Intake', 'Care Load Growth Rate', and rolling averages/standard deviations, providing a deeper understanding of system dynamics.
- **Forecasting Models (Regression):**
 - **Data Leakage Fix:** Initial regression models suffered from data leakage due to rolling features implicitly including the current day's target. This was fixed in the feature engineering code (cell 97bdb5e5) by applying `.shift(1)` to all rolling calculations, ensuring only past data is used. All regression models were subsequently re-evaluated.
 - **Gradient Boosting Regressor:** Achieved the best performance among regression models with MAE of 32.10 and RMSE of 49.21, suggesting it effectively captured the underlying patterns. This model is recommended for its realistic performance metrics.
 - **Random Forest Regressor:** Also performed well with MAE of 44.70 and RMSE of 75.95. This model is also recommended for its realistic performance metrics.
 - **Linear Regression and Huber Regressor:** Despite the data leakage fix, these models still report suspiciously perfect scores (MAE/RMSE of 0.00). This anomaly is potentially attributed to the `fillna(0)` strategy for missing dates creating easily predictable zero-valued data points for these specific linear models, requiring further investigation.
 - **XGBoost Regressor:** Showed MAE of 51.61 and RMSE of 187.25.
 - **SVR:** Performed poorly with MAE of 4836.78 and RMSE of 4937.12, indicating it was not suitable for this dataset with default parameters.
- **Classification Models:**
 - **FutureWarning Fix:** A FutureWarning related to the `multi_class` parameter in LogisticRegression was noted. The Logistic Regression model definition (cell 4f8c3cf6) was updated to wrap LogisticRegression with `OneVsRestClassifier` to explicitly handle multi-class classification and suppress the warning.
 - After addressing the warning, **Random Forest, SVC, Decision Tree, Gradient Boosting, and Gaussian Naive Bayes** emerged as top performers, all achieving an F1-Score of approximately 0.857 and Accuracy of 0.865. These models are effective in classifying the load categories.
- **Future Predictions:** The Gradient Boosting Regressor was retrained on the full historical dataset and used to generate direct forecasts for the next 18 months (approximately 547 days) of 'Total System Load'. The predictions enforce non-negative values and show a plausible future trajectory.
- **Key Visualizations:** Successful generation of various 3D plots, including:
 - A **3D Scatter Plot** for classification model performance, showing model groupings by metrics.

- A **3D Surface Plot** for regression model performance (MAE vs. RMSE), incorporating a "transparent distinguishable wall" to separate metrics.
 - A **3D Heatmap Surface Plot** for classification model performance, visualizing scores across models and metrics.
 - **Dynamic System Load:** Significant fluctuations in 'Total System Load', with rolling averages revealing underlying trends and rolling standard deviation highlighting periods of higher variability.
 - **Pressure Points:** Detected 'prolonged strain windows' where positive net intake coincided with high system load, indicating periods of potential resource strain.
-

Made By Prathamesh Bhurke

Contact Info:

Email: prathameshbhurke666@gmail.com
